А.С. Терлецкий, Е.С. Терлецкая

НЕЙРОННЫЕ СЕТИ И ИСКУСТВЕННЫЙ ИНТЕЛЛЕКТ

Основы нейронных сетей на языке Python

УЧЕБНО-МЕТОДИЧЕСКОЕ ПОСОБИЕ

Липецк

2023

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Министерство просвещения Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Липецкий государственный педагогический университет
имени П.П. Семенова-Тян-Шанского»

А.С. Терлецкий, Е.С. Терлецкая

НЕЙРОННЫЕ СЕТИ И ИСКУСТВЕННЫЙ ИНТЕЛЛЕКТ

Основы нейронных сетей на языке Python

Учебно-методическое

пособие

Липецк
2023

FreeBusta is knowledge
without borders!



УДК 004.8:004.032.6

ББК 32.813

Т 351

Рекомендовано к печати кафедрой информатики, информационных технологий и защиты информации ФГБОУ ВО «Липецкий государственный педагогический университет имени П.П. Семенова-Тян-Шанского».
Протокол № 2 от 21.09.2023 г.

Терлецкий, А.С. Нейронные сети и искусственный интеллект: Основы нейронных сетей на языке Python: учебно-методическое пособие / А.С. Терлецкий, Е.С. Терлецкая. – Липецк: ЛГПУ имени П.П. Семенова-Тян-Шанского, 2023. – 76 с.

ISBN 978-5-907792-40-1

Учебно-методическое пособие «Нейронные сети и искусственный интеллект: основы нейронных сетей на языке Python» входит в состав УМК подготовки бакалавров по направлениям, связанным с IT, такими как: «Прикладная математика и информатика», «Информационные системы и технологии», «Информационная безопасность», «Профессиональное обучение», «Информатика и вычислительная техника», «Педагогическое образование (с двумя профилями подготовки)», «Педагогическое образование (с одним профилем подготовки)».

Задачей пособия является формирование базовых знаний и навыков в области искусственных нейронных сетей, их архитектуры и применения в различных задачах машинного обучения. Методичка включает в себя как теоретический материал, так и практические задания в виде лабораторных работ. Рекомендовано также слушателям программ дополнительного профессионального образования соответствующего направления подготовки.

УДК 004.8:004.032.6
ББК 32.813
Т 351

Рецензенты:

В.С. Зияутдинов, кандидат пед. наук, доцент кафедры естественных и технических наук, Липецкий казачий институт технологий и управления (филиал) ФГБОУ ВО «Московский государственный университет технологий и управления имени К.Г. Разумовского (Первый казачий университет)»

Д.М. Скуднев, кандидат тех. наук, заведующий кафедрой «Информатика, информационные технологии и защита информации» Липецкого государственного педагогического университета имени П.П. Семенова-Тян-Шанского

ISBN 978-5-907792-40-1

© ФГБОУ ВО "Липецкий государственный педагогический университет
имени П.П. Семенова-Тян-Шанского", 2023
© Терлецкий А.С., Терлецкая Е.С., 2023

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 4
ОСНОВЫ НЕЙРОННЫХ СЕТЕЙ 6
ОРГАНИЗАЦИЯ НЕЙРОНОВ В СЕТЬ 16
АРХИТЕКТУРА НЕЙРОННЫХ СЕТЕЙ 20
ПРОЦЕСС ОБУЧЕНИЯ НЕЙРОННЫХ СЕТЕЙ 24
ФУНКЦИЯ ПОТЕРЬ: КАК ИЗМЕРИТЬ ОШИБКУ СЕТИ? 26
АЛГОРИТМ ОБРАТНОГО РАСПРОСТРАНЕНИЯ ОШИБКИ (BACKPROPAGATION) 28
ПРОСТЫЕ ПРИМЕРЫ ПРИМЕНЕНИЯ 30
ЗАКЛЮЧЕНИЕ 33
Лабораторная работа №1 35
МНОГОСЛОЙНЫЙ ПЕРЦЕПТРОН ДЛЯ КЛАССИФИКАЦИИ РУКОПИСНЫХ ЦИФР
Лабораторная работа №2 44
ПРИМЕНЕНИЕ СВЕРТОЧНЫХ НЕЙРОННЫХ СЕТЕЙ (CNN) ДЛЯ КЛАССИФИКАЦИИ ИЗОБРАЖЕНИЙ ИЗ НАБОРА ДАННЫХ CIFAR-10
Лабораторная работа №3 51
ГЕНЕРАЦИЯ ТЕКСТА С ИСПОЛЬЗОВАНИЕМ LSTM
Лабораторная работа №4 57
GAN ДЛЯ ГЕНЕРАЦИИ ИЗОБРАЖЕНИЙ
Список рекомендуемых источников 67
Приложение 69

FreeBusta is knowledge without borders!

ВВЕДЕНИЕ

Я предполагаю, что к концу XX века использование слов и общих утверждений, что машины думают, не будет необоснованным предвзятым мнением.
Алан Тьюринг

В настоящее время профессиональное владение навыками разработки и применения нейронных сетей и искусственного интеллекта (ИИ) становится всё более востребованным и актуальным в различных отраслях науки и промышленности. Нейронные сети – это инструменты, позволяющие машинам "учиться" на данных и выполнять задачи, которые раньше требовали человеческого вмешательства – от распознавания изображений до анализа сложных данных.

Искусственный интеллект – это обширное поле науки, которое занимается созданием машин, способных выполнять задачи, требующие человеческого интеллекта. В то время как нейронные сети представляют собой один из ключевых компонентов искусственного интеллекта, позволяя системам обрабатывать огромные объемы данных и принимать решения на их основе [1, 2, 4].

Искусственные нейронные сети вдохновлены структурой и функционированием человеческого мозга и благодаря современным технологиям и алгоритмам их возможности продолжают расти. Сегодня нейронные сети используются в медицине, финансах, автомобильной промышленности, играх и многих других сферах, что делает их изучение весьма привлекательным направлением для специалистов в области IT и данных.

Настоящее методическое пособие предназначено как для начинающих исследователей и разработчиков в области искусственного интеллекта, так и для тех, кто хочет погрузиться глубже в тему нейронных сетей. Через данное пособие читатель познакомится с базовыми концепциями, алгоритмами и инструментами для работы с нейронными сетями, а также с практическими примерами их применения.

Рекомендации по использованию данного пособия:

1) **начните с основ.** Прежде чем переходить к более сложным темам искусственного интеллекта и глубокого обучения, убедитесь, что вы твердо освоили базовые концепции, такие как алгоритмы обучения, структуры данных и математические основы нейронных сетей;
2) **практикуйтесь.** Работа с искусственным интеллектом и нейронными сетями требует практики. Регулярно выполняйте упражнения из пособия и сталкивайтесь с реальными задачами;
3) **работайте с кодом.** Изучайте примеры кода, приведенные в пособии, чтобы понимать структуру и логику создания нейронных сетей. Это поможет вам понять, как оптимизировать свои модели.
4) **используйте инструменты отладки.** При работе с искусственным интеллектом важно быстро находить и исправлять ошибки. Инструменты отладки помогут вам в этом;

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

5) **общайтесь с коллегами.** Обмен опытом с другими специалистами в области искусственного интеллекта может предоставить вам новые идеи и решения для ваших задач;
6) **пишите код с нуля.** Научитесь создавать свои нейронные сети, не полагаясь на готовые решения. Это позволит вам лучше понять принципы работы ИИ;
7) **не бойтесь ошибок.** Ошибки в области ИИ – это возможность учиться. Анализируйте свои промахи и извлекайте из них уроки;
8) **следуйте структуре пособия.** Рекомендуется придерживаться последовательности материалов, представленных в пособии. Также уделяйте внимание практическим примерам и упражнениям, которые помогут вам закрепить знания;
9) **ориентируйтесь на официальные источники.** Для глубокого понимания ИИ и нейронных сетей обратитесь к академическим ресурсам и официальной документации по используемым инструментам и фреймворкам. Можно ориентироваться на следующие источники:
    - https://keras.io
    - https://scikit-learn.org/stable/
    - https://pytorch.org
    - https://www.tensorflow.org/

В заключение отметим, что овладение навыками работы с искусственным интеллектом и нейронными сетями позволяет разработчикам создавать продвинутые, адаптивные и интуитивные приложения. Это дает возможность обеспечивать глубокий анализ данных, автоматизировать рутинные задачи и предоставлять пользователю уникальный опыт взаимодействия с системой. Помимо этого, знание методов искусственного интеллекта способствует использованию современных фреймворков и библиотек для создания масштабируемых и высокопроизводительных решений.

FreeBusta is knowledge
without borders!

ОСНОВЫ НЕЙРОННЫХ СЕТЕЙ

Нейронные сети – это концепт, зародившийся на стыке биологии и математики и призванный воссоздать основные принципы работы человеческого мозга в формате математических моделей. Сама идея моделирования мозга датируется ещё докомпьютерной эпохой, когда учёные пытались понять, как наш мозг производит различные вычисления и обрабатывает информацию.

Название "нейронные сети" происходит от того, что мозг человека состоит из множества нейронов – базовых функциональных единиц нервной системы, которые соединяются в сложные сети и образуют нервную ткань.

Биологический нейрон – это клетка, способная принимать сигналы от других нейронов, обрабатывать их и передавать далее. Эта передача информации происходит благодаря синапсам – специфическим структурам, позволяющим нейронам обмениваться электрохимическими сигналами.

Когда речь идёт о математической модели, нейронные сети представляют собой систему, где отдельные "нейроны" или узлы соединены в определённой структуре и обмениваются данными. В таком контексте "нейрон" – это некая функция, которая принимает на вход вектор данных, производит вычисления и выдаёт результат на выходе.

Стоит подчеркнуть, что, несмотря на некоторые аналогии, искусственные нейронные сети далеки от полной имитации биологических процессов. Например, моделирование аксонов, дендритов и их множественных свойств (рис. 1) в искусственных системах является чрезвычайно упрощенным. Тем не менее, идея, что сложные вычислительные задачи могут быть решены коллективным взаимодействием простых вычислительных узлов, остаётся основной в концепции нейронных сетей.

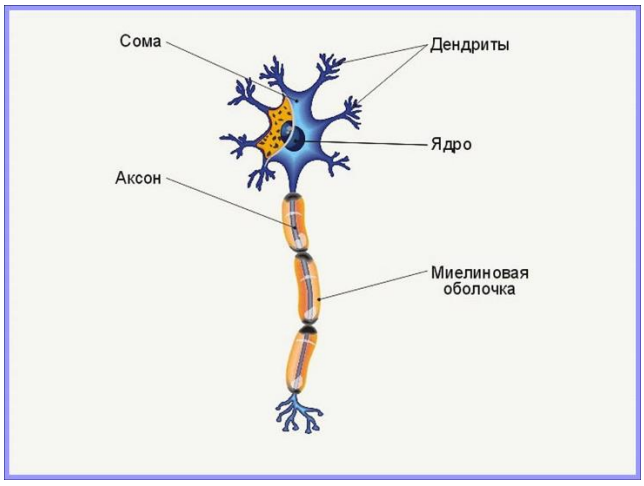


Рисунок 1 – Строение биологического нейрона

Таким образом, нейронные сети, будучи вдохновлены биологической структурой мозга, представляют собой попытку математически моделировать его работу. Этот подход позволяет создавать системы, способные обучаться, адаптироваться и обрабатывать информацию, имитируя, хотя и очень условно, работу человеческого мозга.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

В центре принципа работы нейронных сетей лежит понимание того, как отдельные элементы, или искусственные нейроны, соединены и взаимодействуют друг с другом. Если биологический нейрон может быть активирован стимулом и передавать сигнал через синапсы к другим нейронам, то искусственный нейрон принимает входные данные, производит их обработку по определённой функции и передаёт результат следующему уровню или слою сети.

Но что же заставляет искусственные нейронные сети "обучаться"? Как и в биологических системах, где усиление или ослабление синаптической связи (сила связи между двумя нейронами) определяется опытом, в искусственных системах важную роль играют веса связей. Эти веса меняются в процессе обучения сети на основе данных, что позволяет модели адаптироваться и улучшать свою производительность.

Принцип обратного распространения ошибки стоит в основе многих алгоритмов обучения нейронных сетей. Идея заключается в том, чтобы сначала производить расчёты в прямом направлении от входного слоя к выходному, определяя ошибку на выходе, а затем распространять эту ошибку обратно, корректируя веса связей между нейронами, чтобы минимизировать ошибку в следующих расчётах.

Подобное взаимодействие и постоянная адаптация делают нейронные сети выдающимися инструментами для различных задач, таких как распознавание образов, прогнозирование, рекомендательные системы и многие другие. Нейронные сети способны обнаруживать сложные зависимости в данных, которые могут остаться незамеченными для других методов анализа.

Современные достижения в области нейронных сетей во многом обусловлены ростом вычислительных возможностей, а также доступом к огромным объёмам данных. Современные архитектуры, такие как глубокие нейронные сети (Deep Neural Networks), позволяют обрабатывать информацию на многих уровнях сложности, начиная от первичных признаков и заканчивая сложными абстракциями.

Разговор о нейронных сетях неразрывно связан с пониманием структуры искусственного нейрона – базовой единицы, из которой состоит сеть. Погрузимся в анализ этой структуры.

Искусственный нейрон представляет собой математическую модель, вдохновленную биологическим нейроном. В своем простейшем виде он состоит из ряда входов, каждый из которых ассоциируется с определенным весом. Веса представляют собой коэффициенты, которые определяют степень влияния каждого входного значения на выходное значение нейрона. Эти веса адаптируются и корректируются в процессе обучения сети.

В мире машинного обучения и, в частности, нейронных сетей, существует множество понятий и терминов, которые образуют основу этой дисциплины. Рассматривая концепцию модели, можно представить её как математическое отображение от входных данных к выходным. Это может быть как простая линейная зависимость, так и сложная структура, как глубокая нейронная сеть. Чтобы "настроить" эту модель, необходимо использовать специальный набор данных, называемый обучающим *датасетом*. С его помощью корректируются

FreeBusta is knowledge
without borders!

параметры модели таким образом, чтобы она наилучшим образом описывала зависимости в данных.

Однако, обучая модель только на одном наборе данных, мы рискуем столкнуться с проблемой *переобучения*. Это происходит, когда модель начинает "запоминать" обучающие данные, а не "понимать" общие закономерности. Для проверки качества модели используют тестовый датасет, представляющий собой данные, на которых модель ранее не обучалась. Чтобы бороться с переобучением, применяют различные методы регуляризации, ограничивающие сложность модели.

Центральное место в обучении нейронных сетей занимает *функция потерь*. Она показывает, насколько сильно предсказания модели отличаются от истинных значений. Именно её минимизация и является основной задачей при обучении. Для эффективного обновления параметров модели, стремясь уменьшить значение функции потерь, используют алгоритм обратного распространения ошибки. Этот метод позволяет вычислить, насколько каждый параметр модели влияет на ошибку, и соответственно, корректировать его.

Помимо входов и весов, искусственный нейрон также содержит порог активации, иногда называемый смещением. Этот порог определяет, при каком уровне комбинированного входного сигнала нейрон будет активирован.

После комбинации входных значений с их соответствующими весами и учета порога активации общее значение передается функции активации. Функция активации играет ключевую роль в определении ответа нейрона на комбинированный входной сигнал. Есть множество различных функций активации, включая сигмоидную, гиперболический тангенс, ReLU и многие другие. Выбор конкретной функции зависит от характера задачи и требований к модели.

Важной деталью нейронной сети является *активационная функция*, которая определяет выходное значение каждого нейрона. Она придает нелинейность модели, что позволяет ей выявлять более сложные закономерности в данных. *Глубокое обучение*, поддисциплина машинного обучения, использует многослойные нейронные сети для обработки больших объемов данных, выявляя сложные и многомерные зависимости.

Функция активации может быть простой, например, пороговой функцией, которая активирует нейрон, если комбинированный входной сигнал превышает определенное значение, или более сложной, позволяя нейрону интерпретировать и реагировать на более сложные входные паттерны.Важность функции активации не может быть недооценена. Она добавляет нелинейность в модель, что позволяет нейронным сетям обучаться и аппроксимировать сложные функциональные зависимости, которые были бы недостижимы для линейных моделей.

Рассмотрим теперь принцип работы искусственного нейрона в контексте совокупности таких нейронов, формируя таким образом нейронную сеть.

Когда мы говорим о множестве искусственных нейронов, важно понимать, как они соединены и как информация передается между ними. В искусственных нейронных сетях, как и в биологических, нейроны объединены в слои: входной слой, один или несколько скрытых слоев и выходной слой. Входной слой принимает внешние данные и передает их далее через скрытые слои к выходному слою,

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

который, в свою очередь, предоставляет результат работы сети. В процессе передачи информации от одного слоя к другому каждый нейрон в следующем слое обрабатывает комбинированный сигнал от предыдущих нейронов, учитывая свои веса и применяя функцию активации. Это позволяет сети динамически адаптироваться к представляемой информации и обучаться на основе представляемых данных.

Искусственный нейрон может быть представлен как математическая функция, которая комбинирует входные данные с соответствующими весами, а затем применяет к результату функцию активации. Для начала рассмотрим линейную комбинацию входных данных $x_1, x_2, \dots x_n$ с соответствующими весами $\omega_1, \omega_2, \dots \omega_n$:

$$z = \omega_1 x_1 + \omega_2 x_2 + \cdots \omega_n x_n + b,$$

Здесь b представляет собой смещение или порог активации, который позволяет моделировать решающее правило, независимо от входных данных. После вычисления z его значение передается через функцию активации f, чтобы получить выходное значение нейрона:

$$y = f(z)$$

Существует множество функций активации, каждая из которых имеет свои особенности и применяется в зависимости от конкретной задачи. Например, сигмоидная функция, которая ограничивает выходное значение в диапазоне между 0 и 1:

$$f(z) = \frac{1}{1 + e^{-z}}$$

Еще одна популярная функция активации – это гиперболический тангенс, который возвращает значения в диапазоне от -1 до 1:

$$f(z) = \tanh(z)$$

Важность функции активации заключается в том, что она вносит нелинейность в выходные значения нейронов, что позволяет нейронной сети аппроксимировать сложные нелинейные функции.

В математическом представлении искусственного нейрона ключевое значение имеют веса и порог. Веса ($\omega_1, \omega_2, \dots \omega_n$) определяют степень влияния каждого входного значения на выходное значение нейрона. Пороговое значение b, или смещение, контролирует, при каком уровне комбинированного входного сигнала нейрон будет активирован. Этот процесс можно сравнить с решающим правилом: если комбинированный весовой сигнал превышает определенный порог, нейрон активируется, иначе остается неактивным. Как было ранее упомянуто, после получения комбинированного взвешенного сигнала b он пропускается через функцию активации. Эта функция добавляет нелинейность в модель, что критически важно для возможности нейронных сетей моделировать сложные зависимости.

FreeBusta is knowledge
without borders!

В области глубокого обучения функции активации играют ключевую роль, они вносят нелинейность, позволяя моделям обучаться и представлять более сложные зависимости в данных. Одной из наиболее известных и широко используемых функций активации является Rectified Linear Unit или, как её часто называют, ReLU.

Концепция ReLU основана на идее простого отсечения всех отрицательных значений в функции и оставления положительных значений неизменными. Математически это можно выразить следующим образом:

$$ReLU(x) = \max(0, x)$$

В контексте искусственных нейронных сетей это означает, что нейрон будет активироваться и передавать свой текущий сигнал дальше по сети только в том случае, если входной сигнал положителен. Основное преимущество ReLU заключается в его вычислительной простоте. Поскольку ReLU фактически представляет собой пороговую функцию, она требует минимальных вычислительных ресурсов, что делает ее идеальной для обучения глубоких нейронных сетей.

Однако использование ReLU не лишено проблем. В частности, одной из наиболее часто встречающихся проблем является проблема "умерших" нейронов. Если в процессе обучения веса нейрона настроены таким образом, что нейрон постоянно выводит отрицательные значения, то этот нейрон больше никогда не будет активирован. В результате такой нейрон перестаёт обучаться, что может негативно сказаться на обобщающей способности сети. Для борьбы с этой проблемой были разработаны модификации функции ReLU, такие как Leaky ReLU и Parametric ReLU. В Leaky ReLU отрицательные значения не просто обрезаются на нуле, но умножаются на очень маленькую константу, что позволяет избежать проблемы "умерших" нейронов.

Для наглядности и понимания принципа работы Leaky ReLU рассмотрим её реализацию на Python с использованием библиотек numpy и matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt
# Функция активации ReLU
def relu(x):
    """
    Эта функция принимает на вход вектор чисел и возвращает новый вектор,
    полученный применением функции ReLU к каждому элементу исходного вектора.
    """
    return np.maximum(0, x)
# Генерация данных для визуализации
x = np.linspace(-10, 10, 400)
y = relu(x)
# Визуализация функции активации ReLU
plt.plot(x, y, label='ReLU', color='blue')
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

```
plt.title('ReLU Activation Function')
plt.xlabel('x')
plt.ylabel('ReLU(x)')
plt.grid(True)
plt.legend()
plt.show()
```

Приведенный выше пример состоит из следующих шагов:

1. Импортирование библиотек:
   `numpy` используется для вычислений и работы с массивами,
   `matplotlib.pyplot` для визуализации.
2. Функция `relu(x)`:
   принимает на вход вектор x;
   применяет функцию ReLU к каждому элементу вектора с использованием `np.maximum(0, x)`, который возвращает максимальное значение из двух: 0 или текущее значение элемента.
3. Генерация данных для визуализации:
   `np.linspace(-10, 10, 400)` генерирует 400 равномерно распределенных точек в диапазоне от -10 до 10.
   Эти точки используются в качестве входных данных для функции ReLU.
4. Визуализация:
   с помощью `plt.plot()` рисуется график функции.
   Дополнительные функции, такие как `plt.title()`, `plt.xlabel()`, и `plt.ylabel()` используются для добавления заголовка к графику и подписей к осям.

В результате выполнения этого кода будет отображен график функции ReLU, иллюстрируя, как она ведет себя на разных значениях входных данных (рис. 2).

FreeBusta is knowledge
without borders!



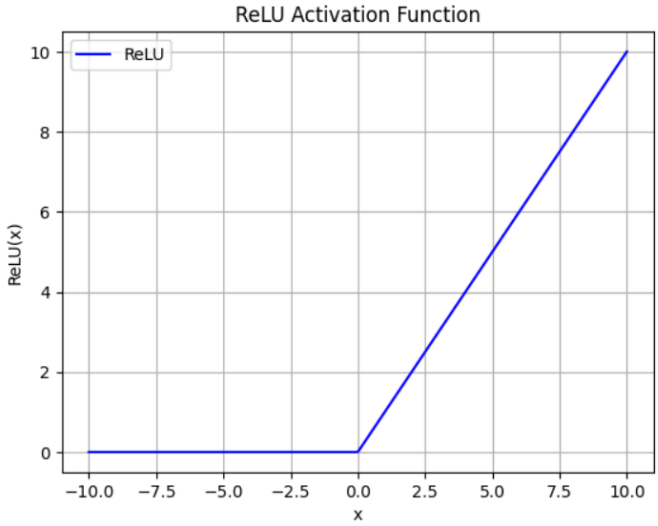


Рисунок 2 – Результат выполнения функции ReLU

Чтобы показать, как работает Leaky ReLU и как эта модификация функции помогает в борьбе с мертвыми нейронами, мы можем начать с визуализации активационных функций. Демонстрация ReLU и Leaky ReLU на одном графике поможет наглядно увидеть разницу между ними (рис. 3).

```
import numpy as np
import matplotlib.pyplot as plt
# ReLU функция активации
def relu(x):
    return np.maximum(0, x)
# Leaky ReLU функция активации
def leaky_relu(x, alpha=0.01):
    return np.where(x > 0, x, x * alpha)
# Генерация данных для визуализации
x = np.linspace(-10, 10, 400)
y_relu = relu(x)
y_leaky_relu = leaky_relu(x)
# Визуализация
plt.plot(x, y_relu, label='ReLU')
plt.plot(x, y_leaky_relu, label='Leaky ReLU', linestyle='--')
plt.title('ReLU vs. Leaky ReLU Activation Functions')
plt.xlabel('x')
plt.ylabel('Activation Function Value')
plt.grid(True)
plt.legend()
plt.show()
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

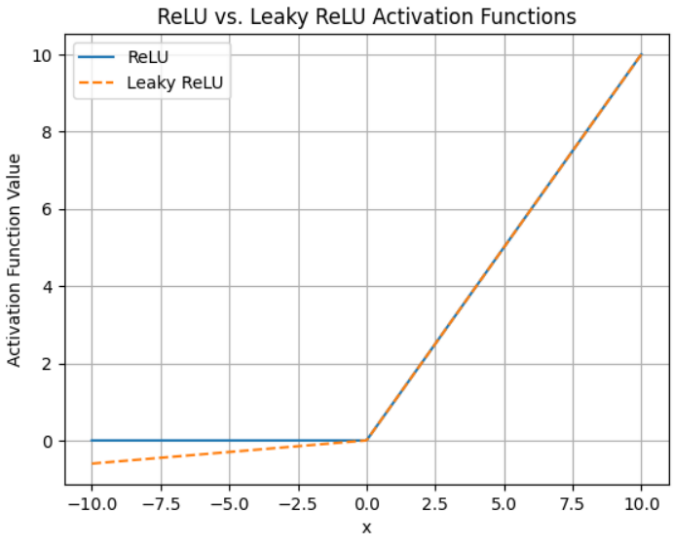


Рисунок 3 – Визуализация работы функции Leaky ReLU

Функции ReLU и Leaky ReLU очень похожи, особенно при положительных значениях x, где они идентичны. Различие между ними становится заметным только при отрицательных значениях x. В то время как ReLU возвращает 0 для всех отрицательных входных данных, Leaky ReLU возвращает значение, которое немного меньше нуля, что достигается путем умножения входного значения на небольшую константу (например, 0.01).

Такое поведение Leaky ReLU иллюстрируется разрывом в линии графика для отрицательных значений x. Хотя этот разрыв может казаться небольшим на графике, в реальных задачах машинного обучения даже такие небольшие отклонения могут иметь большое значение, особенно при обучении глубоких нейронных сетей, где "мертвые" нейроны могут стать серьезной проблемой.

Функция активации ReLU часто используется в глубоком обучении, особенно в сверточных нейросетях (CNN) и глубоких полносвязных нейросетях (DNN). Возьмем популярный пример, где ReLU может быть применена – Сверточная нейросеть для классификации изображений. В этом примере рассмотрим простую CNN для классификации изображений на датасете CIFAR-10 с использованием библиотеки TensorFlow и Keras.

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
Flatten, Dense
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras.preprocessing import image

# Загрузка и предобработка данных
(x_train, y_train), (x_test, y_test) =
cifar10.load_data()
```


FreeBusta is knowledge
without borders!

```
x_train, x_test = x_train / 255.0, x_test / 255.0

# Построение модели
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', in-
put_shape=(32, 32, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10)
])

model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategorical-
Crossentropy(from_logits=True),
metrics=['accuracy'])

# Обучение модели
model.fit(x_train, y_train, epochs=10, valida-
tion_data=(x_test, y_test))

# Функция для загрузки и предобработки изображения
def load_and_preprocess_img(img_path):
    img = image.load_img(img_path, target_size=(32, 32))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    return img_array / 255.0

# Загрузка изображения
img_path = 'frog.png'
img_array = load_and_preprocess_img(img_path)

# Предсказание для изображения
predictions = model.predict(img_array)
predicted_class = np.argmax(predictions, axis=1)

class_names = ['airplane', 'automobile', 'bird', 'cat',
'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

# Вывод результатов
plt.imshow(image.load_img(img_path))
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

```
plt.title(f"Predicted: {class_names[pre-
dicted_class[0]]}")
plt.show()
```

В приведенном выше коде ReLU используется в сверточных и полносвязных слоях модели. После обучения модели обычно следует её оценка на тестовой выборке, а затем использование модели для предсказания на новых данных. Приведенный выше пример выполняет следующую последовательность:

- загрузка данных;
- создание и обучение модели;
- загрузка нового изображения;
- предсказание класса этого изображения;
- визуализация результата.

Процесс обучения нейронной сети для определения объектов на изображении представлено на рисунке 4.

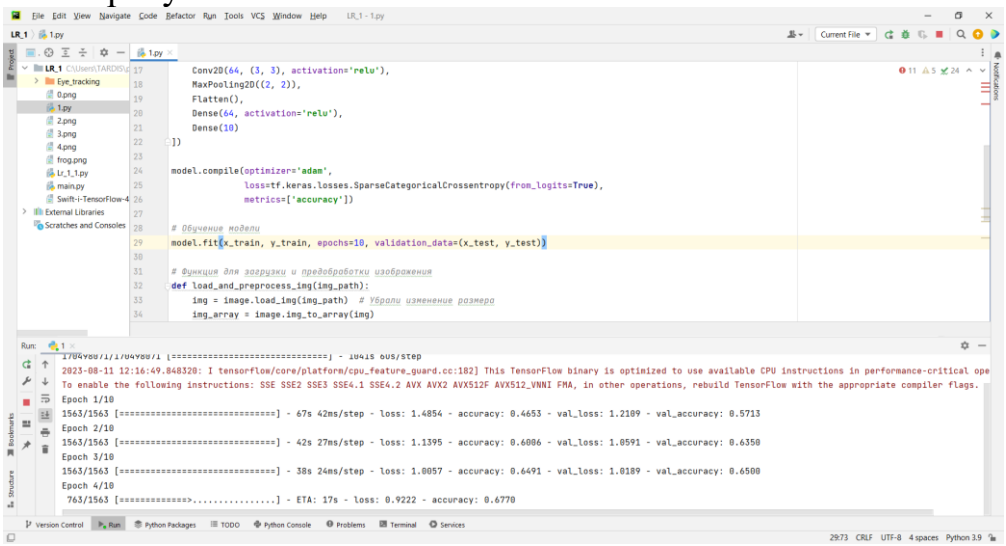


Рисунок 4 – Процесс обучения нейронной сети

В процессе обучения нейронная сеть научилась определять объекты на изображениях с точностью 85,6%, но этот показатель можно увеличить, подняв количество «эпох», во время которых обучается наша нейронная сеть. Для того чтобы не обучать нейронную сеть при каждом запуске скрипта, ее необходимо сохранить добавив, следующий код:

```
# Обучение модели
model.fit(x_train, y_train, epochs=10, valida-
tion_data=(x_test, y_test))
# Сохранение модели после обучения
model.save('cifar10_model.h5')
```

В дальнейшем для загрузки обученной модели необходимо использовать следующий код:

FreeBusta is knowledge
without borders!

```
# Загрузка сохраненной модели
model = load_model('cifar10_model.h5')
```

Эффективность искусственного нейрона определяется его способностью к обучению, а именно коррекции своих весов на основе ошибки между фактическим и ожидаемым выходом. Этот процесс оптимизации часто реализуется с помощью методов градиентного спуска, которые корректируют веса таким образом, чтобы минимизировать общую ошибку сети.

Завершая этот обзор, необходимо подчеркнуть, что хотя искусственные нейроны и биологические нейроны имеют схожие принципы работы, существуют и значительные различия. Если биологический нейрон является чрезвычайно сложной и динамической системой, способной к множеству функций, то искусственный нейрон, в свою очередь, является упрощенной моделью, созданной для определенных задач обработки информации. Тем не менее, в совокупности искусственные нейроны формируют нейронные сети, способные решать сложнейшие задачи и аппроксимировать функции, которые невозможно решить традиционными методами программирования.

ОРГАНИЗАЦИЯ НЕЙРОНОВ В СЕТЬ

Нейронные сети представляют собой сложные структуры, которые вдохновлены строением человеческого мозга. Как и в биологических системах, искусственные нейронные сети состоят из множества элементарных обработчиков - нейронов, которые объединены в определенной структуре. Понимание этой структуры является ключевым для осознания принципов функционирования и обучения нейронных сетей.

Нейронная сеть состоит (рис. 5):

1) входного слоя;
2) скрытого слоя – может состоять из нескольких слоев;
3) выходного слоя.

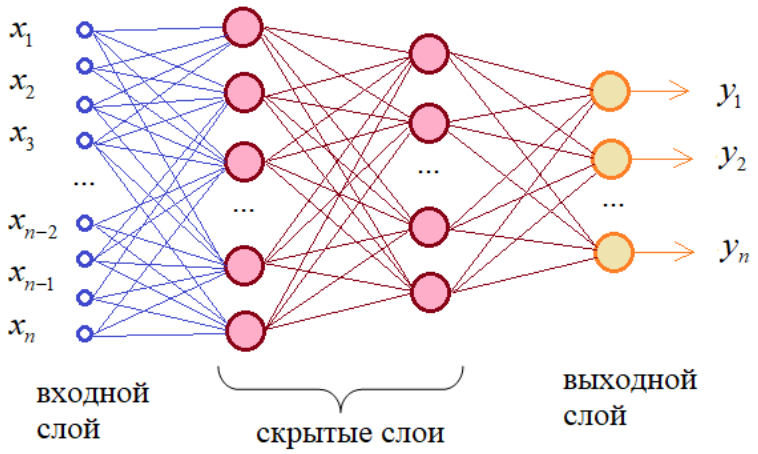


Рисунок 5 – Строение нейронной сети

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Входной слой нейронной сети – это портал, через который внешние данные вводятся в модель для дальнейшего анализа и обработки. Он играет роль интерфейса между внешним миром и сложной внутренней структурой сети.

Основная задача входного слоя заключается в представлении данных в форме, которую сеть может обрабатывать. В отличие от других слоев, входной слой не выполняет никаких математических операций над данными и не содержит функций активации. Его главное назначение – передача данных следующему, так называемому "скрытому", слою. Размерность входного слоя напрямую зависит от характеристик входных данных.

Рассмотрим классический пример с изображениями размером 28x28 пикселей, такими как рукописные цифры из набора данных MNIST. В этом случае входной слой будет иметь 784 нейрона (28 умножить на 28), причем каждый нейрон будет соответствовать одному пикселю изображения. Если это цветное изображение в формате RGB, то необходимо будет 28x28x3 нейрона.

Это изображение представляется в виде массива чисел, где каждое число отражает интенсивность соответствующего пикселя. Эти числа подаются на входной слой сети, и каждый нейрон принимает значение интенсивности отдельного пикселя.

Входной слой – это критически важный компонент нейронной сети. Он определяет, как данные будут представлены внутри сети, и служит отправной точкой для всех последующих слоев и операций. Правильное структурное и концептуальное понимание входного слоя обеспечивает успешное и эффективное проектирование и обучение нейронной сети.

Скрытые слои в контексте нейронных сетей представляют собой сердце модели, обеспечивая большую часть ее вычислительной мощности. Эти слои расположены между входным и выходным слоями и содержат нейроны, которые производят математические операции над входными данными, преобразуя их для получения желаемого результата на выходе.

Скрытые слои выполняют функцию извлечения и комбинирования признаков из входных данных. Через процесс обучения эти слои "узнают", какие комбинации признаков наиболее релевантны для задачи, которую необходимо выполнить, будь то классификация, регрессия или что-то другое.

В процессе обучения нейронной сети веса нейронов в скрытых слоях подстраиваются таким образом, чтобы минимизировать ошибку между предсказанным и реальным выходом. Число скрытых слоев и количество нейронов в каждом слое влияют на "глубину" и "ширину" нейронной сети. Нейронные сети с большим количеством скрытых слоев часто называют "глубокими", что привело к термину "глубокое обучение". Не существует строгих правил относительно количества скрытых слоев или нейронов в них. Однако важно понимать, что увеличение числа нейронов и слоев может улучшить точность модели до определенного предела, после чего может наступить переобучение.

Каждый нейрон в скрытом слое применяет функцию активации к своему входу перед тем, как передать результат следующему слою.

FreeBusta is knowledge
without borders!

Эти функции активации, такие как ReLU (Rectified Linear Unit), сигмоида или гиперболический тангенс, вносят нелинейность в модель. Благодаря этой нелинейности нейронные сети могут аппроксимировать очень сложные функции и решать нетривиальные задачи.

Представим, что входные данные – это набор пикселей изображения с кошкой, а задача сети – определить, что это изображение кошки. Первый скрытый слой может извлекать базовые признаки, такие как грани и контуры. Следующий слой может комбинировать эти грани для определения более сложных структур, например, формы ушей или глаз. Еще один слой может собирать эти структуры вместе для определения облика кошки.

Скрытые слои играют центральную роль в способности нейронных сетей обрабатывать и интерпретировать данные. Они преобразуют входные данные через ряд математических операций и функций активации, извлекая и комбинируя ключевые признаки для получения результата на выходе. Правильная конфигурация и настройка скрытых слоев – ключевой момент в проектировании эффективных и точных нейронных сетей.

В выходном слое нейронной сети заканчивается процесс обработки данных. После прохождения через входные и скрытые слои, информация достигает выходного слоя, который преобразует эту информацию в конечный результат, соответствующий задаче, которую необходимо решить.

На практике выходной слой олицетворяет собой то, ради чего создавалась нейронная сеть. Если рассмотреть пример с классификацией изображений, где целью является определение, какой объект изображен на фотографии, выходной слой будет генерировать вероятности принадлежности изображения к каждому из возможных классов. При этом функция активации softmax может использоваться для того, чтобы гарантировать, что сумма всех вероятностей равна одному.

В других случаях, например, при решении задачи регрессии, где целью является предсказание непрерывной величины, выходной слой может просто возвращать численное значение. Так, при прогнозировании цены товара выходной слой возвращает предсказанную стоимость.

Однако независимо от конкретной задачи важно понимать, что выходной слой не работает изолированно. Он зависит от результатов, полученных на предыдущих этапах обработки, и является последним этапом, на котором происходит преобразование данных в необходимый формат.

Выбор правильной функции активации для выходного слоя также играет критически важную роль. Например, сигмоидная функция активации часто используется в задачах бинарной классификации, так как она возвращает значение между 0 и 1, что можно интерпретировать как вероятность. В то время как для задач многоклассовой классификации функция softmax становится предпочтительней, поскольку она возвращает набор вероятностей для каждого класса.

Таким образом, выходной слой не просто формирует ответ нейронной сети. Он предоставляет интерпретируемый и полезный результат, который можно использовать в реальных приложениях и задачах.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Давайте разберем код, который иллюстрирует создание нейронной сети, включая входной, скрытый и выходной слои. Для этого можно использовать библиотеки `tensorflow` и `keras`.

1. Входной слой:

Вначале определяется входной слой. Здесь мы используем изображения размером 32x32 с тремя каналами (RGB). Слой Input определяет форму входных данных:

```
import tensorflow as tf
from tensorflow.keras.layers import Input
input_layer = Input(shape=(32, 32, 3))
```

2. Скрытые слои:

После входного слоя следуют один или несколько скрытых слоев. Для примера рассмотрим два сверточных слоя и два слоя пулинга. Эти слои проходят через изображение с помощью маленьких "окон" и изучают признаки изображения на разных уровнях детализации. В нашем коде первый сверточный слой изучает 32 различных набора признаков (или "фильтров"), а второй слой изучает 64.

Слои пулинга (MaxPooling2D) уменьшают размерность изображений, выбирая наиболее выраженные признаки из предыдущего сверточного слоя. Они помогают делать модель более эффективной и устойчивой к незначительным изменениям в изображении:

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D
# Первый сверточный слой с 32 фильтрами размером 3x3 и
активационной функцией ReLU
hidden_layer_1 = Conv2D(32, (3, 3), activa-
tion='relu')(input_layer)
# Первый слой пулинга с размером окна 2x2
hidden_layer_2 = MaxPooling2D((2, 2))(hidden_layer_1)
# Второй сверточный слой с 64 фильтрами размером 3x3 и
активационной функцией ReLU
hidden_layer_3 = Conv2D(64, (3, 3), activa-
tion='relu')(hidden_layer_2)
# Второй слой пулинга с размером окна 2x2
hidden_layer_4 = MaxPooling2D((2, 2))(hidden_layer_3)
```

3. Выходной слой:

На этом этапе необходимо определить выходной слой нейронной сети. Для задачи многоклассовой классификации (например, 10 классов) можно использовать полносвязный слой с 10 нейронами и активационной функцией softmax:

```
from tensorflow.keras.layers import Flatten, Dense
flattened = Flatten()(hidden_layer_4)
output_layer = Dense(10, activation='softmax')(flattened)
```

Объединяя все эти слои, мы можем создать модель:

FreeBusta is knowledge
without borders!



```
from tensorflow.keras.models import Model
model = Model(inputs=input_layer, outputs=output_layer)
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Обучая эту модель на датасете CIFAR-10, который включает в себя изображения, относящиеся к 10 разным классам, таким как "самолет", "автомобиль" или "птица", вы сможете достигнуть стадии, когда модель способна анализировать новое изображение и определять его принадлежность к одному из этих 10 классов.

АРХИТЕКТУРА НЕЙРОННЫХ СЕТЕЙ

При проектировании нейронной сети важно выбрать правильную архитектуру. Простые нейронные сети, такие как перцептрон, состоят только из одного или нескольких слоев и имеют ограниченную способность моделирования.

Перцептрон – это одна из самых ранних форм нейронных сетей. Он состоит из одного или нескольких входных нейронов, одного выходного нейрона и, возможно, нескольких скрытых нейронов. Основная идея перцептрона заключается в том, чтобы взять входные данные, взвесить их и выдать предсказание.

С момента создания перцептрона было разработано множество других архитектур, каждая из которых предназначена для решения определенных задач. Например, сверточные нейронные сети идеально подходят для обработки изображений, в то время как рекуррентные сети спроектированы для работы с временными последовательностями.

Полносвязные нейронные сети представляют собой фундаментальную архитектуру в области глубокого обучения. Они получили свое название из-за того, что каждый нейрон в одном слое связан с каждым нейроном в следующем слое. Эти соединения обычно имеют веса, которые определяют степень влияния одного нейрона на другой.

Основное предназначение плотно связанных сетей – это обработка входных данных и их преобразование в выходные определенной формы. При этом входные данные "проходят" через ряд слоев, каждый из которых преобразует данные с помощью активационной функции.

Для понимания структуры полносвязной сети представьте себе дверной глазок, через который вы пытаетесь разглядеть приходящего к вам человека. Если смотреть напрямую, вы видите лицо человека целиком, но если посмотреть через глазок, вы увидите лишь фрагменты или абстракции этого лица. Точно так же нейронные сети "смотрят" на входные данные, проводя их через ряд преобразований, чтобы выявить важные для задачи характеристики.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

В контексте машинного обучения входные данные могут быть чем угодно: числами, изображениями, текстом или звуком. После того как эти данные подаются на вход полносвязанной сети, они проходят через ряд слоев. Каждый слой состоит из нейронов, которые применяют активационные функции к данным, преобразуя их перед передачей на следующий слой.

Такие сети способны обучаться, оптимизируя веса своих соединений на основе различий между предсказанными и реальными данными. Обучение, как правило, происходит с использованием алгоритма обратного распространения ошибки.

На выходе такой сети мы получаем результат, который может быть использован для классификации, регрессии или любой другой задачи. Например, если нам нужно определить, является ли изображение изображением кошки или собаки, последний слой сети будет предоставлять вероятности для каждого из этих классов. Таким образом, плотно связанные нейронные сети играют ключевую роль в мире глубокого обучения, позволяя преобразовывать сложные входные данные в понятные и полезные выходные данные.

Сверточные нейронные сети (CNN) являются вариацией обычных нейронных сетей и особенно эффективны для обработки структурированных данных на сетке, таких как изображения. Они революционировали область компьютерного зрения за счет своей способности автоматически и адаптивно изучать пространственные иерархии признаков из изображений. Суть нейронной сети такого типа заключается в использовании сверток вместо обычных полносвязных слоев. Это позволяет сети автоматически и адаптивно учить пространственные иерархии признаков.

Свертка в математическом контексте – это операция, применяемая к двум функциям, которая производит третью функцию. В контексте CNN свертка применяется к входным данным и ядру (или фильтру) свертки. Ядро "скользит" по входному изображению, чтобы произвести карту признаков, выявляя определенные пространственные характеристики на изображении.

После сверточного слоя часто следует слой пулинга (pooling layer), который уменьшает размерность данных. Самый распространенный метод пулинга – это максимальный пулинг, где из подмножества данных выбирается максимальное значение.

После нескольких сверточных и пулинговых слоев CNN часто используют один или несколько полносвязанных слоев для классификации или регрессии. Эти слои анализируют высокоуровневые признаки, изученные сверточными слоями, для выполнения конечной задачи. Как и другие нейронные сети, CNN обучается с использованием обратного распространения ошибки и градиентного спуска. Ошибка рассчитанная на выходном слое, распространяется обратно через сеть, чтобы обновить веса.

Сверточные нейронные сети имеют множество применений, но они особенно сильны в задачах компьютерного зрения, таких как классификация изображений, распознавание объектов, анализ видео и многое другое.

FreeBusta is knowledge
without borders!



Сети с пропусками (Skip-connection networks), также известные как сети с остаточными соединениями или остаточные сети, представляют собой архитектуры, в которых выходы одного или нескольких слоев передаются напрямую в более глубокие слои сети. Эта концепция была впервые представлена в работе по остаточным сетям (ResNet) и с тех пор стала стандартным подходом для многих глубоких архитектур нейронных сетей.

Проблема исчезающего градиента давно является препятствием для эффективного обучения глубоких нейронных сетей. Суть проблемы заключается в том, что градиенты, которые распространяются обратно по сети в процессе обучения, могут становиться очень маленькими, что делает обновление весов в начальных слоях практически невозможным. Сети с пропусками решают эту проблему, предоставляя "путь наименьшего сопротивления" для градиентов, позволяя им протекать напрямую через пропуски.

В классической архитектуре остаточной сети слой с пропуском берет выходы из одного слоя и добавляет их к выходам из более глубокого слоя. Это обычно реализуется с помощью простого оператора сложения. Например, если у нас есть выходы из слоя L и слоя L+2, вместо того чтобы просто передавать выходы слоя L+ 2 дальше по сети, мы сначала добавляем к ним выходы из слоя L.

Рассмотрим задачу классификации изображений. Представьте, что у нас есть глубокая нейронная сеть, состоящая из 50 слоев. В традиционной архитектуре выходы из каждого слоя передаются следующему слою последовательно. Однако в сети с пропусками выходы, например, из 10-го слоя могут быть переданы напрямую 12-му или 13-му слою, минуя один или два промежуточных слоя.

Остаточные сети и другие архитектуры с пропусками показали значительные улучшения в производительности на многих задачах машинного обучения. Они позволяют обучать глубокие сети без сталкивания с проблемами, связанными с исчезающим градиентом, и таким образом способствуют улучшению обобщающей способности модели.

Рекуррентные нейронные сети (RNN) являются особой категорией нейронных сетей, разработанных специально для работы с последовательностями данных. В отличие от традиционных нейронных сетей, которые предполагают независимость между входами, RNN способны использовать свою внутреннюю память для обработки последовательностей, что делает их особенно полезными для таких задач, как анализ временных рядов, обработка естественного языка и распознавание речи. Сердцем рекуррентной сети является идея повторного использования выходов нейронов в качестве дополнительных входных данных для следующего шага. Этот подход позволяет RNN улавливать зависимости в данных, которые проявляются через время или порядок, и эффективно интегрировать информацию из предыдущих шагов.

На практике, когда рекуррентная сеть обрабатывает последовательность, она итеративно проходит по каждому элементу последовательности. На каждом шаге RNN обновляет свое скрытое состояние на основе предыдущего скрытого состояния и текущего входа.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Это скрытое состояние служит внутренней памятью сети и используется для кодирования информации о предыдущих шагах.

Однако классические RNN столкнулись с проблемами при обучении долгосрочных зависимостей в данных из-за проблемы исчезающего и взрывного градиента. Эти проблемы привели к разработке новых архитектур, таких как LSTM (долгая краткосрочная память) и GRU (вентилируемые рекуррентные блоки). Эти модификации вводят специализированные механизмы в структуру RNN, чтобы эффективно управлять потоком информации и градиентами на протяжении длительных последовательностей.

Рекуррентные нейронные сети нашли широкое применение в различных областях. В области обработки естественного языка они использовались для задач машинного перевода, генерации текста и семантического анализа. В аудио- и видеообработке RNN могут быть применены для распознавания речи или прогнозирования следующего кадра в видеоряде.

Капсульные сети (Capsule Networks) представляют собой инновационную архитектуру нейронных сетей, которая была предложена Джеффри Хинтоном и его коллегами в 2017 году. Основной идеей этой архитектуры является попытка решить некоторые проблемы, связанные со сверточными нейронными сетями (CNN), в частности, их чрезмерную чувствительность к различным вариациям и деформациям на изображениях.

Вместо того чтобы полагаться только на скалярные величины, капсульные сети используют векторы для представления информации. Эти векторы, называемые "капсулами", содержат информацию не только о вероятности присутствия какой-либо особенности, но и о ее текущем состоянии. Такой подход позволяет капсульным сетям быть более устойчивыми к различным изменениям в данных.

Один из ключевых аспектов капсульных сетей – это механизм динамического маршрутизирования. Вместо того чтобы жестко фиксировать пути, по которым информация проходит через сеть (как в CNN), капсульные сети определяют маршруты на основе данных. Таким образом, если определенная особенность на изображении важна для последующего слоя, информация от этой особенности будет направлена именно туда. Благодаря своей структуре капсульные сети могут сохранять иерархические отношения между частями объекта, даже если объект был искажен или повернут.

Традиционные CNN часто требуют обширной аугментации данных для улучшения производительности, в то время как капсульные сети менее чувствительны к мелким изменениям во входных данных. Векторные представления в капсулах могут предоставить более ясное понимание того, что сеть "видит" или "учит".

Несмотря на многие потенциальные преимущества, капсульные сети до сих пор остаются предметом исследований. Они требуют более сложных вычислений и могут быть менее эффективными по сравнению с оптимизированными сверточными сетями в некоторых приложениях.

FreeBusta is knowledge
without borders!

Также стоит отметить, что на данный момент наиболее сложные задачи, такие как глубокое обучение на больших наборах данных, до сих пор успешно решаются с помощью сверточных сетей.

ПРОЦЕСС ОБУЧЕНИЯ НЕЙРОННЫХ СЕТЕЙ

Обучение нейронных сетей является итеративным процессом оптимизации, цель которого – минимизировать разницу между предсказанными выходами сети и истинными метками обучающих данных. Основой этого процесса является метод градиентного спуска, который корректирует веса сети так, чтобы уменьшить ошибку предсказания.

На высоком уровне обучение нейронной сети происходит в следующем порядке:

Прямой проход: в начале каждой итерации сети принимают входные данные и проводят их через все свои слои, генерируя прогноз на выходе. Этот прогноз основан на текущем состоянии весов и смещений сети. Например, для изображения кошки, проходящего через сверточную нейронную сеть, прогноз может быть вероятностью того, что изображение действительно представляет собой кошку.

Расчет потерь: после получения прогноза необходимо оценить, насколько точным он является. Это делается с помощью функции потерь, которая измеряет разницу между прогнозом и истинной меткой. Например, если истинная метка для изображения кошки равна 1, а прогноз сети равен 0,8, ошибка будет равна 0,2.

Обратный проход: после расчета потерь происходит обратный проход, который является, пожалуй, самой критической частью обучения нейронной сети. В этом этапе вычисляется градиент функции потерь по отношению к каждому весу сети. Суть в том, чтобы понять, как каждый вес влияет на конечную ошибку, и в каком направлении его нужно корректировать.

Корректировка весов: с учетом рассчитанных градиентов происходит корректировка весов. Веса сети обновляются в направлении, обратном градиенту, чтобы уменьшить ошибку. Скорость, с которой происходят эти изменения, регулируется параметром, известным как скорость обучения.

Процесс затем повторяется для множества итераций или эпох, пока ошибка не станет приемлемо малой или пока не будет достигнут другой критерий остановки.

В качестве примера рассмотрим обучение нейронной сети для задачи регрессии. Представьте, что у нас есть данные о продажах товара в зависимости от рекламного бюджета. Наша цель – обучить сеть предсказывать продажи на основе заданного бюджета. В процессе обучения сети ей предъявляются различные значения бюджета, и она делает предсказания.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Ошибки между этими прогнозами и реальными данными используются для корректировки весов сети, делая ее прогнозы все более и более точными с каждой итерацией.

Одной из проблем в обучении нейронных сетей является переобучение, когда модель становится "слишком хороша" в предсказаниях обучающих данных, но теряет способность к обобщению на новых данных. Для борьбы с этой проблемой используются методы регуляризации.

Один из популярных методов – это L1 и L2 регуляризация, которые добавляют штраф к функции потерь на основе величины весов. Другой распространенный метод – Dropout, при котором на каждой итерации случайным образом "отключаются" определенные нейроны, что заставляет сеть становиться менее зависимой от каких-либо конкретных нейронов и более устойчивой к переобучению.

Также важно использовать валидационные наборы данных для мониторинга производительности сети на протяжении процесса обучения. Если производительность на валидационных данных начинает ухудшаться, это может быть признаком переобучения, и обучение может быть остановлено или изменено.

Обычно обучение нейронной сети происходит не на всем наборе данных за один раз, а на меньших подмножествах, называемых батчами. После того как сеть видела все обучающие примеры один раз, говорят, что одна эпоха завершена. Обычно требуется множество эпох, чтобы сеть достигла приемлемой производительности.

Например, предположим, что у нас есть 1000 обучающих примеров и мы используем размер батча 100. Таким образом, на каждую эпоху будет 10 итераций. На каждой итерации происходит прямой и обратный проход, и веса обновляются. Этот процесс повторяется до тех пор, пока не будет достигнут заданный критерий остановки, например, после определенного числа эпох или когда потеря достигнет определенного порога.

Обучение с учителем – это метод машинного обучения, при котором модель обучается на основе пар "входные данные - целевая переменная", используя эталонные ответы. Этот процесс можно представить как обучение с применением "учителя", который указывает модели, какие ошибки она совершает, и корректирует её поведение до тех пор, пока она не начнёт давать правильные предсказания.

При обучении с учителем в начале процесса у нас имеется обучающий набор данных, который содержит входные данные и соответствующие им правильные ответы. Наша задача – настроить параметры модели так, чтобы минимизировать разницу между предсказанными ею ответами и эталонными значениями из набора данных. Разницу (или "ошибку") между предсказанными и истинными значениями часто оценивают с помощью функции потерь. Например, в задачах регрессии часто используется среднеквадратичная ошибка, а в задачах классификации – перекрестная энтропия.

Рассмотрим классическую задачу обучения с учителем – линейную регрессию. Пусть у нас есть набор данных о продажах товара в зависимости от затрат

FreeBusta is knowledge
without borders!

на рекламу. Наша цель – предсказать продажи на основе суммы, потраченной на рекламу.

Входные данные и целевая переменная: в этом случае входными данными являются затраты на рекламу, а целевой переменной – объем продаж.

Модель: мы предполагаем, что зависимость между затратами и продажами можно описать линейной функцией. Модель будет иметь параметры – веса, которые определяют угол наклона и смещение линии.

Функция потерь: для оценки точности нашей модели используется среднеквадратичная ошибка.

Оптимизация: используя метод градиентного спуска, мы корректируем веса модели таким образом, чтобы минимизировать функцию потерь.

После успешного завершения обучения модель можно использовать для предсказания продаж на основе новых данных о затратах на рекламу.

Одним из главных преимуществ обучения с учителем является его прямолинейность и интерпретируемость. Если у нас есть достаточное количество качественных обучающих данных, этот метод может быть очень эффективным. Однако он также требует четкого определения целевой переменной и наличия эталонных данных для обучения.

Результаты обучения с учителем напрямую зависят от качества и полноты обучающего набора данных. Если данные смещены или содержат ошибки, это может негативно сказаться на качестве обученной модели.

ФУНКЦИЯ ПОТЕРЬ: КАК ИЗМЕРИТЬ ОШИБКУ СЕТИ?

Ключевым элементом в обучении нейронных сетей и многих других моделей машинного обучения является функция потерь. Эта функция представляет собой математическое выражение, позволяющее оценить разницу между предсказанными значениями и реальными (истинными) значениями. Выбор подходящей функции потерь имеет критическое значение, так как именно на её основе модель корректирует свои параметры с целью минимизации ошибки.

Основные аспекты функции потерь:

1) непосредственная оценка ошибки: функция потерь квантифицирует ошибку модели, чем больше значение функции потерь, тем хуже работает модель, и наоборот;

2) дифференцируемость: в большинстве случаев, особенно когда используется метод градиентного спуска для оптимизации, функция потерь должна быть дифференцируема. Это позволяет вычислять градиенты ошибки по отношению к параметрам модели и, соответственно, корректировать их.

В качестве примера таких функций могут служить:

- среднеквадратичная ошибка (Mean Squared Error, MSE): используется в задачах регрессии. Представляет собой среднее значение квадратов разностей между предсказанными и реальными значениями;
- перекрестная энтропия (Cross-entropy): широко применяется в задачах классификации. Эта функция потерь измеряет разницу между истинным распределением вероятностей и предсказанным моделью.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Для демонстрации функций потерь на Python можно использовать TensorFlow, так как он является одним из наиболее популярных и широко используемых инструментов для работы с нейронными сетями. Демонстрация использования функций на языке python представлена ниже. Результат выполнения программы представлен на рисунке 6.

```
import tensorflow as tf
# Генерируем пример данных
y_true = [3.0, 4.5, 2.8, 5.6]  # Истинные значения
y_pred = [2.9, 4.6, 2.7, 5.5]  # Предсказанные значения
# Среднеквадратичная ошибка (MSE)
mse = tf.keras.losses.MeanSquaredError()
mse_result = mse(y_true, y_pred).numpy()
print(f"Mean Squared Error: {mse_result}")
# Перекрестная энтропия (Cross-entropy)
y_true_probs = [0.9, 0.2, 0.8, 0.3]  # Истинные вероятно-
сти принадлежности к классу 1
y_pred_probs = [0.9, 0.8, 0.8, 0.7]  # Предсказанные ве-
роятности принадлежности к классу 1
ce = tf.keras.losses.BinaryCrossentropy()
ce_result = ce(y_true_probs, y_pred_probs).numpy()
print(f"Cross-entropy: {ce_result}")
```



Рисунок 6 – Результат демонстрации использования функции потерь

1. Среднеквадратичная ошибка (Mean Squared Error, MSE) – это одна из самых популярных функций потерь для регрессионных задач. Она измеряет средний квадрат разности между истинными и предсказанными значениями. Иными словами, MSE определяет, насколько далеко находятся предсказанные значения от истинных. Значение 0,009999981150031 09 означает, что в среднем квадрат разницы между вашими предсказаниями и истинными значениями составляет примерно 0,01. Меньшее значение MSE указывает на лучшее качество модели (хотя это также зависит от масштаба данных).
2. Перекрестная энтропия (Cross-entropy) – это часто используемая функция потерь для задач классификации, она измеряет разницу между истинной вероятностью класса и предсказанной вероятностью этого класса. Чем

FreeBusta is knowledge
without borders!

меньше значение перекрестной энтропии, тем ближе предсказания к истинным меткам. Значение 0,7768616676330566 для перекрестной энтропии говорит о том, что средняя разница между истинными вероятностями и предсказанными вероятностями составляет примерно 0,78. В идеале значение перекрестной энтропии должно быть ближе к нулю; большое значение указывает на то, что модель делает ошибки в предсказаниях.

Стоит заметить, что в данном случае эти метрики были применены к сгенерированным примерам данных, и поэтому их значения не обязательно отражают реальное качество какой-либо модели на реальных данных. Это лишь численные примеры того, как рассчитываются эти функции потерь.

АЛГОРИТМ ОБРАТНОГО РАСПРОСТРАНЕНИЯ ОШИБКИ (BACKPROPAGATION)

Обратное распространение ошибки, часто упоминаемое как "backpropagation", стоит в центре методов обучения искусственных нейронных сетей. Этот метод преследует цель корректировки весов сети в ответ на ошибку, которую сеть допускает при предсказаниях на обучающем наборе данных. Начиная с вычисления ошибки на выходе сети, алгоритм обратного распространения передает эту ошибку обратно, от выходного слоя к входному, уточняя веса на каждом этапе. Такой подход гарантирует, что веса, которые в наибольшей степени влияют на ошибку, корректируются больше остальных.

В ходе этого процесса вначале происходит прямое распространение входных данных через сеть, что приводит к предсказанию сети. Затем это предсказание сравнивается с истинным значением для вычисления ошибки. Ошибка, полученная на этом этапе, представляет собой разницу между ожидаемым и предсказанным значением. Далее, используя производные функций активации и функции потерь, определяется градиент этой ошибки по отношению к весам сети. Градиент в этом контексте указывает на направление и масштаб изменений, которые следует внести в веса, чтобы уменьшить ошибку.

После получения градиентов веса сети корректируются. Корректировка весов выполняется с использованием определенного метода оптимизации, например, метода стохастического градиентного спуска. Этот шаг может повторяться многократно, пока ошибка не станет приемлемо малой или пока не будут выполнены другие критерии остановки.

Рассмотрим простой пример. Представим нейронную сеть, предназначенную для прогнозирования цен на дома на основе их характеристик, таких как площадь и количество комнат. Если после прямого прохождения данных через сеть предсказанная цена дома отличается от известной реальной цены, обратное распространение ошибки позволяет корректировать веса сети таким образом, чтобы уменьшить эту разницу при следующем прохождении.

Давайте рассмотрим простой пример обратного распространения ошибки на языке Python с использованием библиотеки TensorFlow и Keras для создания и обучения простой полносвязной нейронной сети.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Цель нашего примера – обучить нейронную сеть прогнозировать цену дома на основе одной характеристики: площади дома. Разберем по шагам:

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
# Генерация простых данных для обучения
X = np.array([50, 60, 70, 100, 120, 150, 200, 240, 300],
dtype=float)
Y = np.array([100, 120, 140, 220, 260, 330, 450, 520,
660], dtype=float)
# Нормализация данных
X_max = np.max(X)
Y_max = np.max(Y)
X_norm = X / X_max
Y_norm = Y / Y_max
# Создание простой полносвязной модели с одним нейроном
model = keras.Sequential([keras.layers.Dense(units=1,
input_shape=[1])])
# Компиляция модели с использованием метода оптимизации и
функции потерь
model.compile(optimizer='sgd', loss='mean_squared_error')
# Обучение модели
model.fit(X_norm, Y_norm, epochs=1000)
# Прогнозирование цены дома с площадью 85 кв. м
predicted_price_norm = model.predict([85 / X_max])
predicted_price = predicted_price_norm[0][0] * Y_max
print(f"Предсказанная цена для дома площадью 85 кв.м:
{predicted_price} тыс. попугаев")
```

Для лучшей производительности и стабильности нейронных сетей данные часто нормализуются. В приведённом коде выше данные нормализуются делением на максимальное значение в каждом массиве, что приводит их к диапазону между 0 и 1.

В качестве модели используется полносвязная модель с одним нейроном, `input_shape=[1]` указывает на то, что на вход модели подается одно значение (площадь дома). Модель обучается на нормализованных данных с использованием оптимизатора и функции потерь. Задается 1000 эпох, что означает, что модель будет обучаться на данных 1000 раз. Код, представленный в качестве примера, представляет базовую реализацию линейной регрессии с использованием нейронной сети для прогнозирования цен на дома на основе их площади. Результат работы кода представлен на рисунке 7.

FreeBusta is knowledge
without borders!



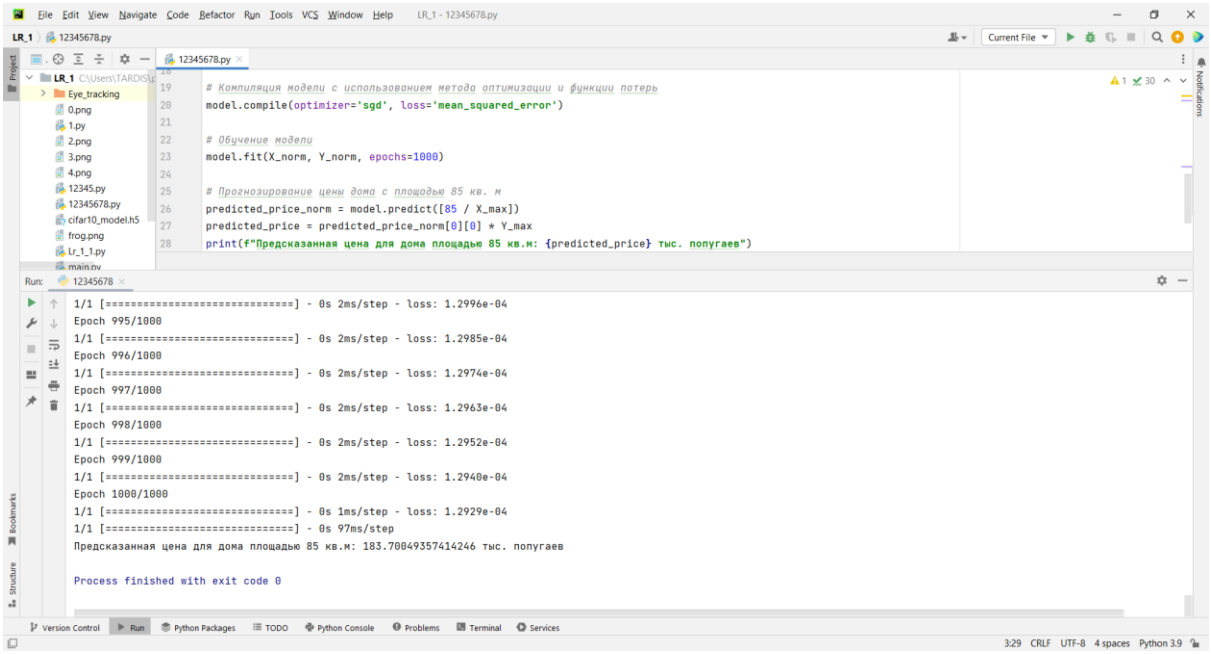


Рисунок 7 – Результат работы нейронной сети по предсказанию цен

ПРОСТЫЕ ПРИМЕРЫ ПРИМЕНЕНИЯ

Применение машинного обучения и, в частности, нейронных сетей, растёт с каждым годом, проникая в различные области науки и бизнеса. Ниже приведены примеры применения этих технологий, иллюстрирующие их широкий спектр возможностей.

В медицине, например, нейронные сети помогают в диагностике заболеваний. Используя данные медицинских изображений, таких как МРТ или рентген, алгоритмы могут автоматически определять наличие или отсутствие патологий, иногда с точностью, превосходящей человеческую. Это позволяет врачам быстрее и эффективнее ставить диагнозы и выбирать оптимальный план лечения.

В области финансов и банковского дела машинное обучение используется для предсказания кредитоспособности клиентов. На основе истории транзакций, данных о доходах, возрасте, образовании и многих других параметров, алгоритмы могут оценивать риски и вероятность невозврата кредита.

Ритейлеры и интернет-магазины применяют машинное обучение для рекомендации товаров своим клиентам. Исходя из истории покупок, просмотров товаров и других интеракций пользователя с платформой, системы рекомендаций предлагают наиболее релевантные товары, что увеличивает вероятность покупки.

В сфере развлечений и медиа, например, на платформах потокового контента, таких как Netflix или Spotify, используются алгоритмы для предложения фильмов, сериалов или музыкальных треков, основываясь на предпочтениях пользователя и интересах.

В области транспорта и автомобильной промышленности машинное обучение стоит в основе разработки систем беспилотного управления. Алгоритмы анализируют информацию от датчиков и камер, позволяя автомобилю

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

принимать решения о маневрах на дороге. Однако не только сложные системы, такие как автономные автомобили, используют машинное обучение. Давайте рассмотрим более простой практический пример, демонстрирующий применение нейронных сетей. В этом примере мы создадим простую модель для выполнения логической операции XOR.

```
import numpy as np
from tensorflow import keras
# Создаем простую модель
model = keras.Sequential([
    keras.layers.Dense(units=10, activation='relu',
input_shape=(2,)),
    keras.layers.Dense(units=1, activation='sigmoid')
])
# Компилируем модель
model.compile(optimizer='adam',
loss='binary_crossentropy', metrics=['accuracy'])
# Тренировочные данные для операции XOR
data = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
labels = np.array([[0], [1], [1], [0]])
# Обучаем модель
model.fit(data, labels, epochs=1000)
# Сохраняем модель для дальнейшего использования
model.save("xor_model.h5")
# Функция для предсказания на вводимых данных
def predict():
    try:
        # Запрашиваем у пользователя входные данные
        a = int(input("Введите первое значение (0 или 1):
"))
        b = int(input("Введите второе значение (0 или 1):
"))
        # Проверяем ввод на корректность
        if a not in [0, 1] or b not in [0, 1]:
            print("Ошибка! Введите только 0 или 1.")
            return
        # Предсказываем значение
        prediction = model.predict(np.array([[a, b]]))
        print(f"Результат XOR для {a} и {b}:
{int(prediction.round())}")
    except ValueError:
        print("Ошибка ввода. Пожалуйста, введите 0 или
1.")
# Вызываем функцию предсказания
predict()
```


FreeBusta is knowledge
without borders!

В приведенном выше примере используются библиотеки `numpy` и `tesorflow (keras)`. Нейронная сеть состоит из двух слоев: первый слой имеет 10 нейронов и функцию активации `relu`, второй слой имеет один нейрон с функцией активации `sigmoid`, что идеально подходит для бинарной классификации. Для обучения модели применяется оптимизатор `adam`, функция потерь `binary_crossentropy` (идеально подходит для бинарной классификации) и метрика `accuracy`, чтобы отслеживать точность модели во время обучения, которое происходит на протяжении 1000 эпох. Функция предсказания вызывается, чтобы пользователь проверил правильность работы сети.

Выше не раз появлялись слова про скорость обучения и оптимизаторы обучения нейронной сети, что же это такое?

Скорость обучения – это один из самых важных гиперпараметров при обучении нейронных сетей. Он определяет, насколько быстро модель обновляет свои веса в ответ на ошибку, которую она делает.

Если представить обучение нейронной сети как спуск по холму (где цель - найти самую низкую точку, то есть минимум функции потерь), то скорость обучения определяет размер шага на каждом этапе спуска.

Слишком большая скорость обучения – модель может "перепрыгивать" минимумы функции потерь и никогда не достигать оптимального решения. Это может привести к нестабильной сходимости или даже расходимости.

Слишком маленькая скорость обучения – обучение может быть очень медленным, и модель может застрять в локальном минимуме, не достигая наилучшего возможного решения.

Оптимизаторы в машинном обучении контролируют, как веса в модели обновляются на основе вычисленной ошибки. Основная цель оптимизатора - минимизировать функцию потерь.

Gradient Descent (Градиентный спуск) – это базовый метод оптимизации. Он обновляет каждый вес на основе градиента функции потерь. Однако этот метод может быть медленным и застревать в локальных минимумах.

Stochastic Gradient Descent (Стохастический градиентный спуск) – это вариант градиентного спуска, который обновляет веса после каждого примера обучения, что делает его более быстрым, но менее стабильным.

Momentum – этот метод учитывает предыдущие шаги градиентного спуска для ускорения сходимости и помогает выйти из локальных минимумов.

Adam (Adaptive Moment Estimation): это один из наиболее популярных оптимизаторов. Он сочетает в себе идеи Momentum и другой метод, называемый RMSprop. Adam автоматически корректирует скорость обучения на основе недавних оценок градиентов.

RMSprop – этот оптимизатор корректирует скорость обучения для каждого веса индивидуально на основе недавних градиентов. Это может быть особенно полезно, когда данные имеют шумы или когда, функция потерь имеет множество плато.

Adagrad – адаптивный градиентный оптимизатор, который также адаптирует скорость обучения для каждого параметра на основе истории градиентов.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Выбор оптимизатора и скорости обучения зависит от конкретной задачи, структуры вашей модели и данных. В большинстве случаев начинают с оптимизатора Adam с его стандартной скоростью обучения, а затем экспериментируют и настраивают оттуда. Однако важно понимать принципы работы различных оптимизаторов, чтобы принимать обоснованные решения в процессе настройки вашей модели.

ЗАКЛЮЧЕНИЕ

Важность понимания основ нейронных сетей не может быть недооценена. В наше время, когда технологии настолько интенсивно развиваются, область глубокого обучения стоит в центре многих инноваций.

Понимание основ нейронных сетей создает основу для дальнейших исследований в области машинного обучения. Это как строить дом: если у вас нет крепкого фундамента, здание не будет крепким. Без понимания базовых принципов работы искусственных нейронов, процесса обратного распространения ошибки и механизмов активации, попытки понять более сложные аспекты такие как сверточные нейронные сети или рекуррентные нейронные сети могут стать чрезмерно запутанными.

Кроме того, важность понимания лежит не только в чисто академическом интересе. В современном мире компании активно ищут специалистов в области глубокого обучения для решения реальных проблем. От автоматизации производственных линий до разработки алгоритмов для самоуправляемых автомобилей знания в этой области становятся ценным ресурсом.

Однако понимание основ – это только начало. Для действительно полного и всестороннего понимания глубокого обучения необходимо двигаться дальше. Повышение знаний часто требует изучения последних исследований, экспериментов на практике и обмена опытом с другими специалистами.

Процесс непрерывного обучения и исследования новых методов и технологий в области глубокого обучения – это путь, который может привести к открытию новых возможностей и горизонтов. Возможно, именно ваше стремление к знаниям приведет к созданию новых инновационных решений, которые изменят мир в лучшую сторону.

Понимание основных принципов нейронных сетей — это, безусловно, важный этап, но это лишь начало пути. Для действительного углубления в эту область и дальнейшего профессионального роста следует принять во внимание несколько ключевых направлений.

Первоначально стоит вникнуть в продвинутые концепции глубокого обучения. Это может включать в себя изучение различных архитектур нейронных сетей, таких как сверточные или рекуррентные сети. Осознание того, как они функционируют и в каких условиях они наиболее эффективны, позволит вам создавать модели, которые могут решать еще более сложные задачи.

Помимо этого, важно держать руку на пульсе современных исследований в области глубокого обучения. Научные журналы, конференции и семинары —

FreeBusta is knowledge
without borders!



отличные ресурсы для погружения в актуальные темы и методы. Возможно, вы обнаружите новые подходы или техники, которые можно будет применить в своей практике или исследованиях.

Опыт практической работы и эксперименты также имеют первостепенное значение. Применять новые подходы, тестировать разные модели на реальных данных, сталкиваться с проблемами и находить их решения — всё это позволяет не только глубже понимать материал, но и расширять свой профессиональный опыт.

Кроме того, сотрудничество и общение с коллегами из области глубокого обучения может быть неоценимым ресурсом. Обмен опытом, дискуссии, совместные проекты или исследования могут открыть новые горизонты и предоставить новые перспективы на уже известные вам концепции.

В заключение, проникновение в глубокое обучение – это непрерывный процесс обучения, исследований и практики. Каждый новый этап, каждая новая задача или проблема позволяют улучшить ваши знания и стать еще более компетентным специалистом в этой динамично развивающейся области.

Источники

1. Браунли, Джейсон. Введение в машинное обучение с помощью Python. Руководство для специалистов по работе с данными [электронный ресурс]/ Джейсон Браунли; пер. с англ. – М.: ООО «И. Д. «Вильямс», 2019. – 212 с. – URL: https://www.labirint.ru/books/595088/ (дата обращения: 23.07.2023).
2. Герон, Орельен. Прикладное машинное обучение с использованием Scikit-Learn и TensorFlow / Орельен Герон; пер. с англ. – М.: ДМК Пресс, 2019. – 542 с.
3. Гудфеллоу, Ян. Глубокое обучение / Ян Гудфеллоу, Йошуа Бенжио, Аарон Курвилль; пер. с англ. – М.: ДМК Пресс, 2018. – 664 с.
4. Карпатий, Андрей. Глубокое обучение. С нуля до профессионала / Андрей Карпатий; пер. с англ. – М.: ДМК Пресс, 2019. – 490 с.
5. Макклур, Тариен. Нейронные сети: учебник для начинающих / Тариен Макклур. – М.: ДМК Пресс, 2020. – 310 с.
6. Николенко, Сергей. Глубокое обучение. Погружение в мир нейронных сетей / Сергей Николенко, Алексей Кадурин, Евгений Архангельская. – М.: ДМК Пресс, 2018. – 456 с.
7. Рашка, Себастьян. Python и машинное обучение [электронный ресурс] / Себастьян Рашка; пер. с англ. – Изд. дом "Питер", 2017. – 418 с. – URL: https://coderbooks.ru/books/python/python_i_mashinnoe_obuchenie_rashka_mirdzhalili_2020/ (дата обращения 15.04.2023).
8. Сайт с открытыми курсами по глубокому обучению Deep Learning Specialization от Эндрю ЫІна. [электронный ресурс] режим доступа: https://www.coursera.org/specializations/deep-learning (дата обращения 15.05.2023).

Забери welcome-бонус 2000 USDT по промокоду FreeBusta

9. Штернцев, Григорий. Нейронные сети: полный курс / Григорий Штернцев. – М.: БХВ-Петербург, 2018. – 528 с.
10. Чоллет, Франсуа. Глубокое обучение на Python [электронный ресурс] / Франсуа Чоллет; пер. с англ. – М.: ООО "И. Д. «Вильямс», 2019. – 384 с. – URL: https://www.litres.ru/book/fransua-sholle-17338303/glubokoe-obuchenie-na-python-pdf-epub-39113884/?yclid=155491749567922175 (дата обращения: 23.07.2023).
11. TensorFlow. Официальная документация и руководства. [электронный ресурс] Режим доступа: https://www.tensorflow.org/overview (дата обращения 15.05.2023).
12. Keras. Официальная документация и руководства. [электронный ресурс] URL: https://keras.io/guides/ (дата обращения 13.04.2023).
13. Neural Network Architectures. Обзор различных архитектур нейронных сетей. [электронный ресурс] Режим доступа: https://neuralnetworksanddeeplearning.com/ (дата обращения 16.05.2023).
14. Fast.ai. Курсы по глубокому обучению. [электронный ресурс] URL: https://course.fast.ai/ (дата обращения 17.04.2023).

Лабораторная работа №1
МНОГОСЛОЙНЫЙ ПЕРЦЕПТРОН ДЛЯ КЛАССИФИКАЦИИ РУКОПИСНЫХ ЦИФР

Цель:

- *познакомиться с основными принципами работы многослойного перцептрона как одного из видов нейронных сетей;*
- *освоить предобработку и нормализацию данных для успешного обучения нейронной сети;*
- *применить знания о структуре и работе нейронной сети для создания собственного многослойного перцептрона с использованием библиотек для глубокого обучения;*
- *изучить процесс обучения нейронной сети, включая выбор оптимизатора и функции потерь;*
- *провести оценку качества работы нейронной сети на тестовых данных, измерить метрики производительности;*
- *проанализировать результаты, полученные после обучения модели, и предложить возможные способы улучшения производительности модели.*

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Многослойный перцептрон (MLP) представляет собой архитектуру искусственных нейронных сетей, в которой узлы (нейроны) организованы в слои. В самом простом случае есть три слоя: входной, скрытый и выходной. Каждый

FreeBusta is knowledge without borders!

нейрон в определенном слое связан с каждым нейроном следующего слоя, и это называется полносвязной архитектурой. Например, входной слой может содержать 784 нейрона, которые соответствуют пикселям в изображении 28x28. В простейшей модели MLP может быть один скрытый слой, содержащий, например, 128 нейронов. Выходной слой содержит столько нейронов, сколько есть классов – в нашем случае 10 нейронов, соответствующих цифрам от 0 до 9.

Каждый входной образец (изображение) преобразуется в вектор значений пикселей. Этот вектор пропускается через сеть, а значения в нейронах обновляются в соответствии с их весами и функцией активации. На выходном слое каждый нейрон содержит вероятность того, что входное изображение относится к классу, которому этот нейрон соответствует. Например, если третий нейрон выдает самую высокую вероятность, модель предсказывает, что входное изображение представляет собой цифру "2".

Активационная функция играет важную роль в нейронных сетях. Она определяет выход нейрона на основе его входных значений. В многослойном перцептроне активационная функция обычно нелинейна. Это позволяет модели улавливать сложные зависимости между данными. Существуют различные типы активационных функций, такие как сигмоидная, гиперболический тангенс и ReLU (Rectified Linear Unit). Выбор активационной функции может влиять на производительность модели.

В нашем случае мы можем использовать функцию ReLU в скрытом слое, так как она быстро считается и хорошо справляется с проблемой затухания градиента. На выходном слое обычно используется функция Softmax, которая преобразует входной вектор в вектор вероятностей, где каждое значение представляет вероятность принадлежности входного образца к определенному классу.

Для оценки эффективности модели используется функция потерь. Она измеряет разницу между предсказанными значениями и истинными значениями. В нашем случае, так как мы решаем задачу классификации, мы можем использовать кросс-энтропийную функцию потерь.

Эта функция минимальна, когда предсказанные вероятности совпадают с истинными значениями, и увеличивается, когда предсказания отличаются от истинных значений. В процессе обучения наша цель – минимизировать значение этой функции, изменяя веса сети.

После того, как мы вычислили значение функции потерь, нам нужно каким-то образом обновить веса в нашей модели, чтобы уменьшить ошибку. Для этого используется алгоритм обратного распространения ошибки. Он начинает с выходного слоя и распространяет ошибку назад по сети, обновляя веса.

Оптимизатор определяет, как именно будут обновляться веса. Самый простой оптимизатор – это стохастический градиентный спуск (SGD). Он просто вычитает градиент функции потерь, умноженный на некоторый коэффициент (скорость обучения), из текущих весов.

Более сложные оптимизаторы, такие как Adam или RMSprop, могут адаптировать скорость обучения в зависимости от обучения, что может ускорить сходимость и улучшить итоговую производительность модели.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

После того, как модель обучена, мы хотим узнать, насколько хорошо она работает. Для этого мы используем тестовый набор данных, который не участвовал в процессе обучения. Мы просто пропускаем эти данные через модель и смотрим, насколько хорошо предсказания совпадают с истинными метками.

Метрика, которую мы используем для оценки эффективности модели в задаче классификации, обычно называется точностью. Она представляет собой процент правильно классифицированных образцов.

В результате, мы имеем полную картину процесса построения и обучения многослойного перцептрона для классификации рукописных цифр.

ПРАКТИЧЕСКАЯ ЧАСТЬ

В практической части для работы мы будем использовать следующие библиотеки:

- tensorflow
- keras
- numpy
- matplotlib

Для этого в терминале IDE необходимо ввести следующие команды:

- `pip install tensorflow`
- `pip install keras`
- `pip install numpy`
- `pip install matplotlib`

При использовании в качестве IDE Aanconda необходимо ввести следующие команды:

- `conda install tensorflow`
- `conda install keras`
- `conda install numpy`
- `conda install matplotlib`

Если при установке библиотек возникают проблемы, то можно попробовать ввести следующие команды. Эта команда гарантирует, что pip соответствует вашей версии Python:

- `python -m pip install tensorflow`
- `python -m pip install keras`
- `python -m pip install numpy`
- `python -m pip install matplotlib`

Что бы проверить правильность установки и версию библиотеки, необходимо ввести следующий код:
`import tensorflow as tf`

FreeBusta is knowledge
without borders!

```
print(tf.__version__)
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flat-
ten
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras import backend as K
```

Реализацию многослойного перцептрона для классификации рукописных цифр можно разделить на несколько пунктов:

1. Импорт библиотек, которые содержат модули для работы с нейронными сетями и данными.

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical
```

Модуль tensorflow предоставляет все основные функции и методы для создания, компиляции и обучения нейронных сетей. Модули tensorflow.keras.datasets, tensorflow.keras.models, tensorflow.keras.layers и tensorflow.keras.utils предоставляют дополнительные инструменты для работы с данными и моделями. Рассмотрим их немного подробнее:

- tensorflow.keras.datasets: этот модуль содержит готовые к использованию наборы данных, которые вы можете загрузить и использовать для обучения и тестирования своих моделей. В вашем случае вы используете набор данных MNIST, который содержит изображения рукописных цифр;
- tensorflow.keras.models: этот модуль содержит классы для создания моделей машинного обучения в TensorFlow. Основные классы включают Model и Sequential. Model – это базовый класс для всех моделей Keras, а Sequential – это класс для создания моделей, состоящих из стека слоев, где каждый слой имеет ровно один входной тензор и один выходной тензор;
- tensorflow.keras.layers: этот модуль предоставляет классы для создания слоев в моделях TensorFlow. Каждый слой представляет собой функцию, которая принимает на вход один или несколько тензоров и возвращает один или несколько тензоров. Некоторые типы слоев, которые вы можете использовать, включают Dense (полносвязный слой), Conv2D (двумерный сверточный слой), MaxPooling2D (двумерный слой максимального пулинга) и многие другие;
- tensorflow.keras.utils: этот модуль предоставляет различные утилиты, которые могут быть полезны при работе с моделями TensorFlow. Один из таких инструментов – функция to_categorical, которую вы использовали для преобразования меток классов из числового формата в формат "one-hot".

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

2. Установка параметров:

```
batch_size = 128
num_classes = 10
epochs = 20
```

Здесь необходимо задать основные параметры обучения:

- `batch_size` – это количество примеров, которые обрабатываются одновременно во время обучения;
- `num_classes` – это количество различных классов в ваших данных, в этом случае 10, так как у вас есть цифры от 0 до 9;
- `epochs` – это количество циклов обучения по всему набору данных.

3. Загрузка данных.

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

В этом шаге необходимо загрузить данные для обучения и тестирования. mnist.load_data() возвращает два набора данных: обучающий (`x_train` и `y_train`) и тестовый (`x_test и y_test`).

- `x_train и x_test` – это изображения цифр;
- `y_train и y_test` – это соответствующие им метки.

4. Преобразование данных.

```
x_train = x_train.reshape(60000, 784)
x_test = x_test.reshape(10000, 784)
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train /= 255
x_test /= 255
```

Во время 4 этапа преобразуются изображения из двумерного формата (28x28 пикселей) в одномерный (784 пикселя). Затем вы преобразуете значения пикселей из целых чисел в диапазоне от 0 до 255 в вещественные числа в диапазоне от 0 до 1. Это делается для того, чтобы упростить обучение нейронной сети.

5. Кодирование меток.

```
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)
```

Здесь преобразуется метки классов из числового формата в формат "one-hot". Например, цифра 3 преобразуется в вектор [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]. Это делается для удобства работы нейронной сети.

6. Создание модели.

```
model = Sequential()
model.add(Dense(128, activation='relu', in-
put_shape=(784,)))
model.add(Dense(num_classes, activation='softmax'))
```


FreeBusta is knowledge
without borders!



На 6 шаге создается архитектура нейронной сети. Ваша сеть будет последовательной (`Sequential`), что означает, что слои добавляются друг за другом. Вы добавляете два слоя - `Dense`, что означает, что все нейроны в слое связаны со всеми нейронами в следующем слое. Первый слой имеет 128 нейронов и функцию активации `ReLU`. Второй слой имеет 10 нейронов (по числу классов) и функцию активации `softmax`, что идеально подходит для многоклассовой классификации.

7. Компиляция модели.

```
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Здесь модель нейронной сети компилируется, необходимо задать функцию потерь, оптимизатор и метрику.

Функция потерь `categorical_crossentropy` – это стандартный выбор для многоклассовой классификации. Оптимизатор `'adam'` – это эффективный метод стохастической оптимизации. Метрика `'accuracy'` позволяет вам отслеживать точность классификации в процессе обучения.

8. Обучение модели.

```
model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs, validation_data=(x_test, y_test))
```

Модель учится предсказывать метки `y_train` по данным `x_train`. Она делает это `epochs` раз, обновляя веса модели с использованием градиентного спуска и оптимизатора, чтобы минимизировать ошибку, вычисленную с помощью функции потерь, также в модель передаются валидационные данные, чтобы модель могла оценить свою эффективность на независимых данных после каждой эпохи.

9. Оценка работы модели

```
score = model.evaluate(x_test, y_test, verbose=0)
print('Test loss:', score[0])
print('Test accuracy:', score[1])
```

После обучения необходимо оценить работу модели на тестовых данных. Это позволяет оценить, насколько хорошо модель будет работать на новых данных, которые она не видела в процессе обучения. В консоль IDE выводится потери (ошибку) и точность тестовых данных (рис. 1).

Полный код к данной лабораторной работе находится в приложении.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

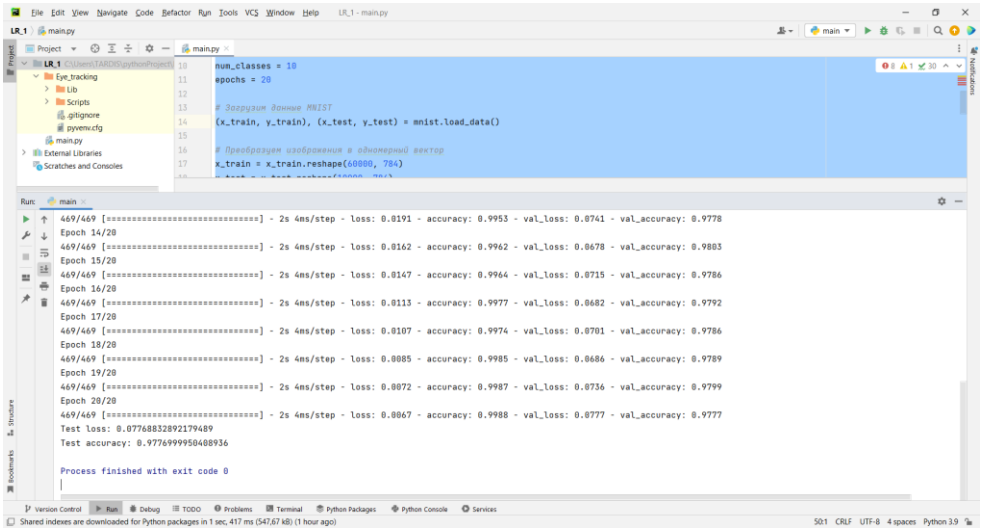


Рисунок 1 – Процесс обучения нейронной сети

После успешного обучения и оценки нашей модели, перейдем к следующему этапу нашей работы – определению конкретных цифр на новых данных. Этот этап демонстрирует практическое применение нашей обученной модели.

Во время обучения и тестирования модель училась распознавать цифры на изображениях из набора данных MNIST, который содержит тысячи изображений рукописных цифр. Теперь же мы применим нашу модель к новым изображениям, которые модель раньше не видела.

Для демонстрации этого мы загрузим новое изображение рукописной цифры, предобработаем его (изменим размер на 28x28 и преобразуем в одномерный вектор), а затем пропустим через нашу модель для предсказания.

Этот процесс демонстрирует, как обученную модель можно использовать в реальных приложениях для распознавания рукописных цифр. Например, такая модель может быть использована для автоматического распознавания почтовых индексов на письмах или формах.

Пожалуйста, обратите внимание, что результаты могут варьироваться в зависимости от качества изображения, сложности рукописного текста и других факторов. В любом случае, результаты этого этапа позволят нам оценить, насколько хорошо наша модель справляется с реальными задачами.

1. Импортируем еще несколько необходимых библиотек:

```
import cv2
import numpy as np
```

2. Зададим путь для тестируемого изображения (рис. 2):

```
image_path = '2.png' # так как изображение лежит в одной директории со скриптом, то путь можно писать таким образом
```

3. Преобразуем тестовое изображение в черно-белый формат:

```
img = cv2.imread(image_path, 0)
```

Здесь используется функция imread из библиотеки OpenCV для чтения изображения. Аргумент 0 означает, что изображение будет загружено в черно-белом формате.

FreeBusta is knowledge without borders!



Рисунок 2 – Тестовое изображение для нейросети

4. Инвертируем цвета:

```
img = cv2.bitwise_not(img)
```

В этой строке выполняется инверсия цветов изображения. Это преобразование нужно для того, чтобы цифры были представлены белым цветом, а фон - черным. Это сделано для соответствия с данными обучения.

5. Изменение размера изображения на 28x28 пикселей:

```
img = cv2.resize(img, (28, 28))
```

Эта строка изменяет размер изображения на 28x28 пикселей, чтобы оно соответствовало формату данных, на которых обучалась модель.

6. Преобразование изображения в одномерный вектор и его нормализация:

```
img = img.reshape(1, 784).astype('float32') / 255
```

Тут изображение преобразуется в одномерный вектор длиной 784 (28x28) и производится его нормализация путем деления на 255 (максимальное значение пикселя в изображении).

7. Применение модели к изображению:

```
prediction = model.predict(img)
```

Этот шаг подразумевает применение обученной модели к нашему предобработанному изображению. В результате получаем массив вероятностей для каждого из 10 классов (цифр от 0 до 9).

8. Получение индекса наиболее вероятного класса:

```
digit = np.argmax(prediction)
```

На этом этапе выбирается класс (цифра), для которого модель предсказала наибольшую вероятность.

9. Вывод предсказанной цифры:

```
print('Predicted digit:', digit)
```

Наконец, в этой строке выводится предсказанная моделью цифра. Это и есть результат работы модели на новом изображении.

Источники:

1. Chollet, F. Глубокое обучение с помощью Python [электронный ресурс] / Франсуа Шолле; пер. с англ. – М.: ДМК Пресс, 2019. – 352 с. – URL:

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

https://www.manning.com/books/deep-learning-with-python (дата обращения: 1.09.2023).
2. Goodfellow, I., Bengio, Y., & Courville, A. Глубокое обучение [электронный ресурс] / Иан Гудфеллоу, Йошуа Бенджио, Аарон Курвилль; пер. с англ. – М.: ДМК Пресс, 2018. – 664 с. – URL: http://www.deeplearningbook.org (дата обращения: 1.09.2023).
3. LeCun, Y., Cortes, C., & Burges, C. База данных рукописных цифр MNIST [электронный ресурс] / Ян ЛеКун, Коринна Кортес, Кристоф Бурдж; пер. с англ. – М.: ДМК Пресс, 2010. – 56 с. – URL: http://yann.lecun.com/exdb/mnist (дата обращения: 1.09.2023).
4. Сайт Keras. Руководство по использованию библиотеки Keras для создания и обучения нейронных сетей. [электронный ресурс] URL: https://keras.io/guides/ (дата обращения: 1.09.2023).
5. Сайт OpenCV. Руководство по использованию библиотеки OpenCV для обработки изображений. [электронный ресурс] URL: https://docs.opencv.org/master/d6/d00/tutorial_py_root.html (дата обращения: 1.09.2023).
6. Сайт TensorFlow. Руководство по созданию и обучению моделей нейронных сетей с использованием библиотеки TensorFlow. [электронный ресурс] URL: https://www.tensorflow.org/tutorials (дата обращения: 1.09.2023).

Задание:

- изучите базу данных MNIST и её структуру. Посмотрите примеры изображений и соответствующих им меток;
- создайте модель нейронной сети с использованием библиотеки TensorFlow. Экспериментируйте с различными структурами и параметрами модели, чтобы достичь наилучшей точности;
- обучите модель на тренировочных данных из базы MNIST. Оцените точность модели на тестовых данных.
- подготовьте изображения рукописных цифр для предсказания их моделью. Эти изображения должны быть различных размеров и цветов. Используйте библиотеку OpenCV для предобработки изображений;
- примените обученную модель для предсказания цифр на подготовленных изображениях. Оцените верность предсказаний;
- измените параметры обучения модели (batch_size, num_classes, epochs) и оцените, как эти изменения влияют на точность модели;
- проведите эксперименты с обучением модели на различных подмножествах данных MNIST (например, используйте только половину тренировочных данных или увеличьте количество тестовых данных) и оцените, как это влияет на ее производительность.

FreeBusta is knowledge without borders!

Лабораторная работа №2

ПРИМЕНЕНИЕ СВЕРТОЧНЫХ НЕЙРОННЫХ СЕТЕЙ (CNN) ДЛЯ КЛАССИФИКАЦИИ ИЗОБРАЖЕНИЙ ИЗ НАБОРА ДАННЫХ CIFAR-10

Цель:

- *познакомиться с принципами работы и применения сверточных нейронных сетей для задачи классификации изображений;*
- *освоить загрузку и нормализацию данных из набора CIFAR-10, приготовив их для обучения модели; применить методы глубокого обучения для создания, загрузки и использования предварительно обученной модели с использованием библиотек TensorFlow и Keras;*
- *изучить процесс предсказания класса изображения на основе данных, поданных на вход модели; провести визуализацию и интерпретацию результатов предсказания, сопоставив их с реальными классами изображений из набора CIFAR-10;*
- *проанализировать возможные пути улучшения и оптимизации представленного кода и модели для повышения её эффективности и точности.*

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Сверточные нейронные сети (CNN) являются разновидностью многослойных перцептронов, основной задачей которых является распознавание паттернов или текстур на изображениях. Они были специально разработаны для этой цели благодаря своей способности искать локальные паттерны на изображении с помощью сверточных слоев. Это ключевое отличие позволяет CNN успешно обучаться на относительно маленьких фрагментах изображений, при этом не теряя важных характеристик объектов.

Для примера можно взять набор данных CIFAR-10. Это коллекция из 60000 изображений разрешением 32x32 пикселя, разделенная на 10 разных классов. Эти классы представляют собой различные объекты, вроде автомобилей, самолетов и животных. Важность предобработки данных не может быть недооценена, особенно в области машинного обучения. В случае изображений из CIFAR-10 критическим шагом предобработки является нормализация значений пикселей, чтобы привести их к диапазону между 0 и 1.

В нашем контексте TensorFlow и Keras играют ведущую роль. TensorFlow – это открытая платформа для машинного обучения от Google, в то время как Keras служит высокоуровневым интерфейсом для создания и обучения моделей глубокого обучения, работающим поверх TensorFlow. Эти инструменты предоставляют набор функций для проектирования, обучения, оценки и деплоя нейронных сетей.

После обучения модель можно использовать для предсказания класса новых изображений. Этот процесс включает в себя подачу изображения на вход модели и дальнейшую интерпретацию результатов, которые представляют собой

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

вероятности принадлежности изображения к каждому из классов. Визуализация этих результатов помогает лучше понять, насколько эффективно модель справляется со своими задачами, и где возможны улучшения.

Сам механизм работы нейронной сети базируется на математических операциях свертки. Когда мы говорим о сверточных нейронных сетях, имеем в виду применение оператора свертки к входным данным. Эта операция объединяет информацию из нескольких входных пикселей, сосредоточиваясь на локальных особенностях изображения. Например, когда рассматривается изображение с различными текстурами и узорами, сверточные слои помогают выделить и агрегировать эту информацию, делая ее более понятной для последующих слоев.

Далее, после сверточных слоев, часто применяются слои пуллинга, которые уменьшают размерность изображения, сохраняя при этом наиболее важные детали. Это помогает уменьшить вычислительную сложность и объем требуемой памяти без существенной потери информации.

Активационные функции в нейронных сетях, такие как ReLU, добавляют нелинейность, что позволяет модели обучаться сложным и неочевидным зависимостям в данных. В контексте сверточных нейронных сетей они могут помочь выявить наиболее ярко выраженные признаки на изображении.

Заключительным этапом является оптимизация, которая корректирует веса модели, учитывая ошибки в предсказаниях. Этот процесс гарантирует, что модель будет постепенно улучшаться, минимизируя разницу между предсказанными и истинными значениями.

Таким образом, внутренние механизмы нейронной сети, будь то сверточные слои, активационные функции или оптимизаторы, служат обеспечению наилучшего качества предсказаний на новых данных.

ПРАКТИЧЕСКАЯ ЧАСТЬ

Итак, мы рассмотрели основы работы с нейронными сетями и процесс классификации изображений. Давайте применим полученные знания на практике и рассмотрим работу CNN на примере кода.

Данный код представляет собой пример классификации изображений с использованием сверточной нейронной сети на наборе данных CIFAR-10 в среде TensorFlow. Чтобы лучше понять механику действий, давайте подробно разберем каждую его часть. Результат работы нейронной сети представлен на рисунке 1, программный код на языке Python приведен ниже.

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential,
load_model
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
Flatten, Dense
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras.preprocessing import image
```


FreeBusta is knowledge
without borders!

```
# Загрузка и предобработка данных
(x_train, y_train), (x_test, y_test) =
cifar10.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

# Загрузка сохраненной модели
#model = load_model('cifar10_model.h5')

#Создение модели
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', in-
put_shape=(32, 32, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Если вы все еще хотите обучать модель, оставьте код
обучения. В противном случае его можно закомментировать:
model.fit(x_train, y_train, epochs=20, valida-
tion_data=(x_test, y_test))

# Сохранение всей модели в файл HDF5
model.save('my_model.h5')

# Функция для загрузки и предобработки изображения
def load_and_preprocess_img(img_path):
    img = image.load_img(img_path, target_size=(32, 32))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    return img_array / 255.0

# Загрузка изображения
img_path = 'sh.png'
img_array = load_and_preprocess_img(img_path)

# Предсказание для изображения
predictions = model.predict(img_array)
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

```
predicted_class = np.argmax(predictions, axis=1)

class_names = ['airplane', 'car', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']
# Вывод результатов
plt.imshow(image.load_img(img_path))
plt.title(f"Predicted: {class_names[pre-
dicted_class[0]]}")
plt.show()
```

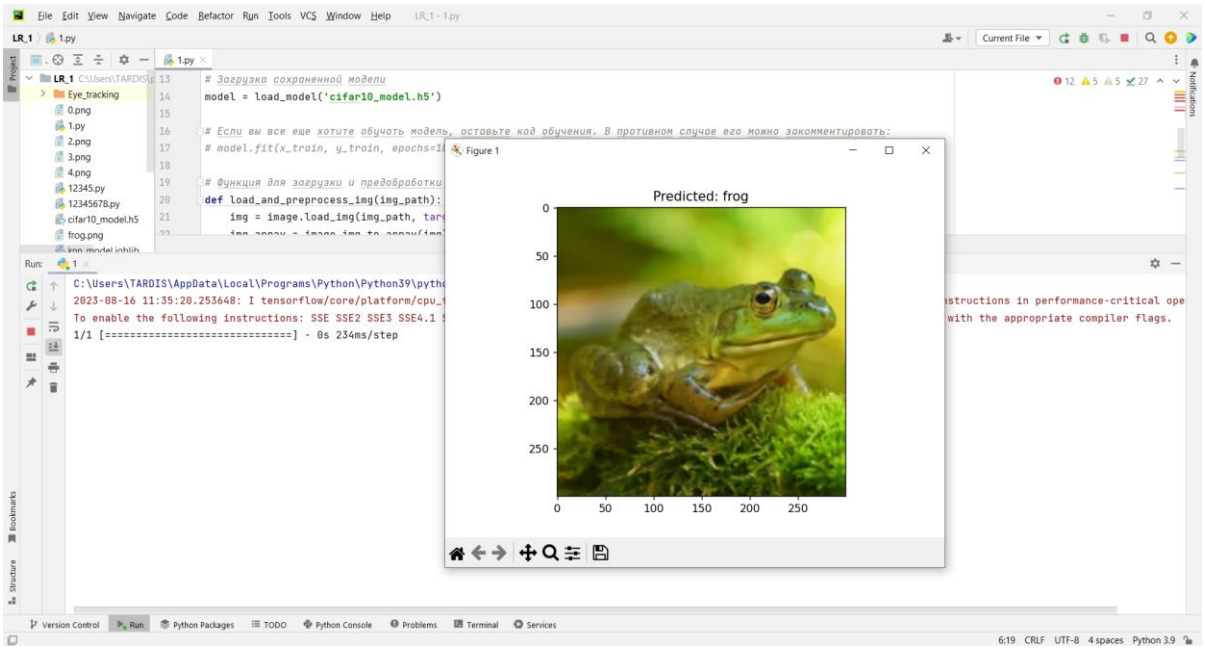


Рисунок 1 – Результат работы нейронной сети по распознаванию изображений

Реализацию данной нейронной сети можно разбить на следующие шаги:

1) импорт необходимых модулей: импортируются модули matplotlib, numpy и библиотека для работы с нейронными сетями TensorFlow;

2) загрузка и предобработка данных: датасет CIFAR-10 включает в себя 10 различных классов изображений. Прежде чем работать с данными, их необходимо загрузить и нормализовать, чтобы значения пикселей были между 0 и 1;

3) загрузка модели: предполагается, что у нас уже есть предварительно обученная модель, которую мы загружаем с помощью функции load_model. Это позволяет экономить время на повторном обучении сети;

4) обучение модели: хотя в этом примере строка обучения закомментирована, она предоставляет возможность дообучить модель, если это потребуется;

5) функция для загрузки новых изображений: чтобы протестировать модель на новых изображениях, написана функция, которая загружает изображение, изменяет его размер до необходимого формата (32x32) и нормализует;

FreeBusta is knowledge
without borders!

6) классификация изображения: здесь загружаемое изображение (в нашем случае frog.png) подается на вход модели, и модель делает предсказание. Предсказанный класс определяется как класс с наибольшей вероятностью;

7) вывод результата: используя matplotlib, мы визуализируем изображение и отображаем предсказанный класс.

Если нам необходимо обучать модель с самого начала, нужно скомпилировать ее, увеличивая ее точность, необходимо добавить запись:

```
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', in-
put_shape=(32, 32, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=10, valida-
tion_data=(x_test, y_test))
```

После обучения модели, необходимо ее сохранить, чтобы в дальнейшем не переобучать ее при каждом запуске скрипта, для этого используется метод save. Этот метод позволяет сохранять архитектуру модели, веса, конфигурацию обучения и состояние оптимизатора. Это означает, что вы можете загрузить модель позже и продолжить обучение с того места, где остановились. Для использования этого метода добавляется запись такого вида:

```
# Сохранение всей модели в файл HDF5
model.save('my_model.h5')
```

Чтобы загрузить модель позже, можно использовать следующий код:

```
from tensorflow.keras.models import load_model
# Загружаем модель из файла
new_model = load_model('my_model.h5')
# Проверяем ее архитектуру
new_model.summary()
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Также стоит отметить, что есть множество способов сохранения модели (только архитектуру, только веса и т.д.), но метод save является наиболее удобным для сохранения всей модели.

Источники:

1. Chollet, F. (2018). Deep Learning with Python. Manning Publications. [электронный ресурс] Доступно по адресу: https://www.manning.com/books/deep-learning-with-python (дата обращения: 1.09.2023).
2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT press. [электронный ресурс] Доступно по адресу: http://www.deeplearningbook.org (дата обращения: 1.09.2023).
3. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). An introduction to statistical learning. Springer. [электронный ресурс] Доступно по адресу: https://www.statlearning.com (дата обращения: 1.09.2023).
4. LeCun, Y., Cortes, C., & Burges, C. (2010). MNIST handwritten digit database. ATT Labs [электронный ресурс]. Available: http://yann.lecun.com/exdb/mnist (дата обращения: 1.09.2023).
5. Сайт Keras. Руководство по использованию библиотеки Keras для создания и обучения нейронных сетей. [электронный ресурс] Доступно по адресу: https://keras.io/guides/ (дата обращения: 1.09.2023).
6. Сайт OpenCV. Руководство по использованию библиотеки OpenCV для обработки изображений. [электронный ресурс] Доступно по адресу: https://docs.opencv.org/master/d6/d00/tutorial_py_root.html (дата обращения: 1.09.2023).
7. Сайт TensorFlow. Руководство по созданию и обучению моделей нейронных сетей с использованием библиотеки TensorFlow. [электронный ресурс] Доступно по адресу: https://www.tensorflow.org/tutorials (дата обращения: 1.09.2023).

Задание:

- изучите базу данных CIFAR-10 и её структуру. Просмотрите примеры изображений и соответствующих им меток. Ознакомьтесь с особенностями каждого из десяти классов (например, автомобиль, птица, лошадь и т. д.);
- создайте свою модель сверточной нейронной сети с использованием библиотеки TensorFlow. Отталкиваясь от предоставленного кода, добавьте или измените некоторые из слоев, чтобы экспериментировать с различными структурами и параметрами. Цель – достичь лучшей возможной точности;
- оцените работу базовой модели. Загрузите уже обученную модель из файла 'cifar10_model.h5' и оцените её производительность на тестовых данных из CIFAR-10, это можно сделать, применив пример приведенного кода и сохранив обученную модель;
- подготовьте внешние изображения для предсказания с помощью модели. Эти изображения могут быть любых размеров и относиться к любому из десяти

FreeBusta is knowledge without borders!

классов CIFAR-10. Вы можете использовать как реальные изображения, так и изображения из Интернета. Используйте предоставленную функцию load_and_preprocess_img для подготовки каждого изображения;

- получите предсказания для своих изображений. Загрузите внешние изображения и используйте обученную модель для получения предсказаний классов. Оцените, насколько точно модель определила классы ваших изображений;
- измените параметры сохраненной модели. На основе предоставленного кода попробуйте изменить некоторые из параметров, таких как количество фильтров в сверточных слоях или количество нейронов в полносвязных слоях, чтобы увидеть, как это влияет на результаты предсказания;
- экспериментируйте с разными подмножествами данных CIFAR-10. Например, попробуйте обучить модель только на части обучающих данных или измените соотношение обучающих и тестовых данных. Оцените, как эти изменения влияют на эффективность модели.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Лабораторная работа №3

ГЕНЕРАЦИЯ ТЕКСТА С ИСПОЛЬЗОВАНИЕМ LSTM

Цель:

- *познакомиться с принципами работы и применения долгосрочной, краткосрочной памяти (LSTM) для задачи генерации текста;*
- *освоить методы предобработки и токенизации литературных текстов, подготовив их для обучения рекуррентной нейронной сети;*
- *применить методы глубокого обучения для создания, тренировки и использования модели LSTM с использованием библиотек TensorFlow и Keras;*
- *изучить процесс генерации новых текстовых фрагментов на основе обученной модели, экспериментируя с различными начальными условиями и параметрами генерации;*
- *провести анализ качества и стилистики сгенерированных текстов, сопоставив их с исходным материалом;*
- *проанализировать возможные пути улучшения и оптимизации представленного кода и модели для улучшения качества и разнообразия генерируемого текста.*

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

В последние годы в области искусственного интеллекта и машинного обучения наблюдается активное развитие методов глубокого обучения. Эти методы стали особенно популярными и эффективными для работы с последовательностями данных, такими как тексты, речь и временные ряды. В центре этого развития стоят рекуррентные нейронные сети (RNN) и их модификация — LSTM (долгосрочная, краткосрочная память).

Рекуррентные нейронные сети выделяются способностью учитывать предыдущую информацию при обработке текущего ввода, что делает их весьма подходящими для анализа текстов и других последовательностей данных. Однако классические RNN столкнулись с проблемой затухания градиента, ограничивая их возможность улавливать долгосрочные зависимости.

Для решения этой проблемы была разработана архитектура LSTM. Она обладает специализированной структурой, включая "ячейку состояния", которая сохраняет информацию на длительные промежутки времени. Вентили LSTM, такие как входной, выходной и забывающий, регулируют поток этой информации, что делает LSTM особенно подходящим для задач, где необходимо улавливать долгосрочные зависимости в данных.

В области генерации текста LSTM сети обучаются на больших объемах текстовых данных, стремясь предсказать следующий элемент последовательности на основе предыдущих. После обучения модель может генерировать новые тексты, сохраняя стиль исходного материала, будь то литературные произведения или другие тексты. Качество генерации зависит от множества факторов, включая объем и качество обучающего набора данных, архитектуру сети и

FreeBusta is knowledge
without borders!

параметры обучения. Эффективное использование LSTM требует экспериментирования и тонкой настройки параметров, что в свою очередь может значительно повлиять на результаты генерации.

В последние годы возросло использование LSTM в различных областях искусственного интеллекта, кроме генерации текста. Они играют ключевую роль в распознавании речи, преобразуя аудиозаписи в текст с учетом контекста из предыдущих слов. В мире машинного перевода эти сети значительно улучшили качество перевода, учитывая контекст и долгосрочные зависимости между словами и фразами. Их применение также расширилось на анализ временных рядов в таких областях как финансы и метеорология, где последовательность данных имеет критическое значение.

Анализ настроений в тексте, особенно в отзывах и комментариях в социальных медиа, стал еще более точным с использованием LSTM, что позволило маркетологам получить лучший взгляд на потребительское мнение. В сфере искусства LSTM начали применять для создания музыкальных композиций, обучая их на основе существующих музыкальных произведений. В области видеоанализа LSTM способны определять активность и предсказывать последующие действия, что нашло применение в системах безопасности.

С каждым годом, благодаря их уникальной способности улавливать информацию из последовательностей данных, LSTM продолжают расширять свое присутствие в различных сферах искусственного интеллекта. Из-за этой уникальной способности, на практике реализация LSTM стала довольно простой благодаря современным фреймворкам глубокого обучения, таким как Keras. Взгляните на базовый пример реализации:

```
from keras.models import Sequential
from keras.layers import LSTM, Dense
```

Здесь мы импортируем необходимые модули из библиотеки Keras. Sequential обозначает, что наша модель будет последовательной, то есть слои будут идти один за другим. LSTM и Dense – это типы слоев, которые мы будем использовать.

```
model = Sequential()
```

На этом шаге мы инициализируем модель как последовательную.

```
model.add(LSTM(128, input_shape=(timesteps, data_dim)))
```

Затем мы добавляем слой LSTM в нашу модель. В этом слое есть 128 единиц (или нейронов). Параметр `input_shape` определяет форму входных данных. В этом случае у нас есть определенное количество временных интервалов (или шагов) `timesteps` и определенная размерность данных `data_dim` на каждом шаге.

```
model.add(Dense(10, activation='softmax'))
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

После LSTM слоя, мы добавляем полносвязный слой (Dense) с 10 нейронами. Этот слой может, например, представлять 10 классов, к которым мы хотели бы классифицировать наши данные. Функция активации softmax обеспечивает вероятностное распределение на выходе, делая сумму всех выходов равной 1.

```
model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Наконец, мы компилируем нашу модель. Мы указываем функцию потерь, которая определяет, насколько хорошо наша модель предсказывает правильные ответы. В данном случае мы используем categorical_crossentropy для многоклассовой классификации. Оптимизатор adam – это метод обновления весов в процессе обучения, и metrics указывает, какие метрики (в данном случае точность) мы хотим отслеживать во время обучения.

Этот код предоставляет базовую структуру для обучения LSTM на любом наборе последовательных данных. Подобные структуры находят применение в разнообразных задачах искусственного интеллекта, от анализа текста до временных рядов и за пределами. Полностью код с комментариями можно посмотреть в приложении.

ПРАКТИЧЕСКАЯ ЧАСТЬ

В современной области искусственного интеллекта генерация текста при помощи глубокого обучения открывает широкие возможности для создания новых алгоритмов и методов обработки данных. Особый интерес представляет использование LSTM, которое благодаря своим уникальным свойствам стоит в авангарде этой области.

На первом этапе нашей работы основное внимание уделяется подготовке данных. Это критически важный момент, так как именно корректность и аккуратность этого этапа определяют качество дальнейшей работы модели. Нам необходимо преобразовать символы текста в численный формат, который будет понятен нашей нейросети.

Далее мы переходим к созданию модели. На этом этапе проектирования архитектуры сети ставится задача выбора оптимального количества и типа слоев, а также других параметров. Этот этап требует особого внимания и опыта, так как от правильного выбора архитектуры зависит эффективность всей системы.

После того, как архитектура определена, начинается этап обучения модели. Используя подготовленные на первом этапе данные, мы "учим" нашу модель распознавать и генерировать новые текстовые последовательности. Этот процесс может занять значительное время, и его успешное завершение станет подтверждением правильности выбора начальных параметров и подготовленных данных.

Завершающим этапом становится генерация текста. Используя обученную модель и начальный фрагмент текста, который служит затравкой, мы создаем новые текстовые последовательности, анализируя их на соответствие исходным данным.

FreeBusta is knowledge without borders!

Теперь, имея перед собой этапы проектирования и понимание их значимости, предлагаем обратить внимание на код, который позволит реализовать вышеописанные принципы на практике.

1. Этап подготовки данных: перед нами стоит задача преобразовать символы текста в численный формат, понятный для обработки нейросетью. Этот этап начинается с загрузки исходного текста и определения уникальных символов в нём:

```
text = open("path_to_your_text_file.txt",
"r").read().lower()
chars = sorted(list(set(text)))
char_indices = dict((c, i) for i, c in enumerate(chars))
indices_char = dict((i, c) for i, c in enumerate(chars))
```

Если вы встретитесь с сообщением об ошибке, представленном на рисунке 1, необходимо указать кодировку используемого файла при его открытии:

```
text = open("text.txt", "r", encoding="utf-
8").read().lower()
```

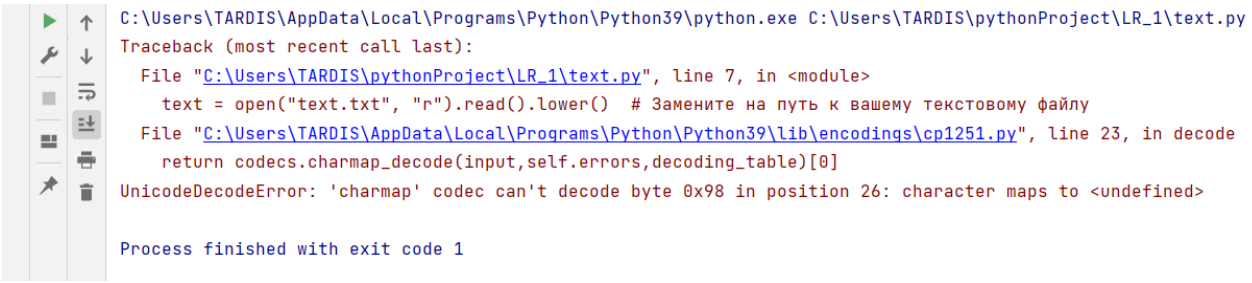


Рисунок 1 – Сообщение об ошибке использования кодировки файла

Для того что бы узнать кодировку файла, можно воспользоваться следующим кодом:

```
import chardet
with open("text.txt", "rb") as f:
    result = chardet.detect(f.read())
print(result['encoding'])
```

Далее формируются обучающие последовательности и соответствующие им целевые символы:

```
maxlen = 40
step = 3
sequences = []
next_chars = []

for i in range(0, len(text) - maxlen, step):
    sequences.append(text[i: i + maxlen])
    next_chars.append(text[i + maxlen])
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

И эти последовательности преобразуются в бинарные матрицы:

```
X = np.zeros((len(sequences), maxlen, len(chars)),
dtype=np.bool)
y = np.zeros((len(sequences), len(chars)), dtype=np.bool)
for i, sequence in enumerate(sequences):
    for t, char in enumerate(sequence):
        X[i, t, char_indices[char]] = 1
    y[i, char_indices[next_chars[i]]] = 1
```

2. Этап создания модели: на этом этапе строится архитектура нейросети. В нашем случае это простая LSTM модель:

```
model = Sequential()
model.add(LSTM(128, input_shape=(maxlen, len(chars))))
model.add(Dense(len(chars), activation='softmax'))
model.compile(loss='categorical_crossentropy', opti-
mizer='adam')
```

3. Этап обучения модели: мы начинаем "учить" модель распознавать и генерировать новые текстовые последовательности:

```
model.fit(X, y, batch_size=128, epochs=10)
```

4. Этап генерации текста: используя обученную модель и начальный фрагмент текста, создаем новые текстовые последовательности:

```
def generate_text(seed_text, length=400):
    generated = seed_text
    for i in range(length):
        x_pred = np.zeros((1, maxlen, len(chars)))
        for t, char in enumerate(seed_text):
            x_pred[0, t, char_indices[char]] = 1.
        preds = model.predict(x_pred, verbose=0)[0]
        next_index = np.argmax(preds)
        next_char = indices_char[next_index]
        seed_text = seed_text[1:] + next_char
        generated += next_char
    return generated
start_index = np.random.randint(0, len(text) - maxlen - 1)
seed_text = text[start_index: start_index + maxlen]
print(generate_text(seed_text))
```

Таким образом, следуя вышеуказанным этапам, можно создать функциональную систему для генерации текста на базе нейросетей LSTM. Полностью код с подробными комментариями представлен в приложении.

FreeBusta is knowledge
without borders!

Источники:

1. Chollet, F. (2018). Deep Learning with Python. Manning Publications. [электронный ресурс] Доступно по адресу: https://www.manning.com/books/deep-learning-with-python (дата обращения: 27.08.2023).
2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT press. [электронный ресурс] Доступно по адресу: http://www.deeplearningbook.org (дата обращения: 27.08.2023).
3. Karpathy, A. (2015). The Unreasonable Effectiveness of Recurrent Neural Networks. [электронный ресурс] Доступно по адресу: http://karpathy.github.io/2015/05/21/rnn-effectiveness/ (дата обращения: 27.08.2023).
4. Brownlee, J. (2018). How to Develop a Character-Based Neural Language Model. Machine Learning Mastery. [электронный ресурс] Доступно по адресу: https://machinelearningmastery.com/develop-character-based-neural-language-model-keras/ (дата обращения: 27.08.2023).
5. Сайт TensorFlow. Руководство по использованию TensorFlow и Keras для создания и обучения нейронных сетей. [электронный ресурс] Доступно по адресу: https://www.tensorflow.org/guide/keras (дата обращения: 27.08.2023).
6. Сайт NumPy. Руководство по использованию библиотеки NumPy для научных вычислений на Python. [электронный ресурс] Доступно по адресу: https://numpy.org/doc/ (дата обращения: 27.08.2023).

Задание:

- изучите предоставленный текстовый файл. Проанализируйте его структуру, содержание и стиль;
- запустите предоставленный код для генерации текста на основе LSTM с использованием библиотеки TensorFlow. Убедитесь в его работоспособности;
- экспериментируйте с длиной входной последовательности (maxlen), изменяя её. Попробуйте значения 20, 40, 60 и 80, и оцените влияние этого параметра на качество генерируемого текста;
- расширьте архитектуру сети, добавив дополнительные слои LSTM перед полносвязным слоем. Сравните результаты генерации текста с базовой моделью;
- экспериментируйте с параметрами обучения, такими как размер пакета и количество эпох. Оцените, как изменение этих параметров влияет на качество генерируемого текста;
- попробуйте различные оптимизаторы, например, 'sgd' или 'rmsprop', вместо 'adam'. Оцените их влияние на результаты генерации текста;
- загрузите другой текстовый файл (например, литературное произведение) и примените к нему вашу модель. Сравните результаты генерации текста для различных исходных данных.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Лабораторная работа №4

GAN ДЛЯ ГЕНЕРАЦИИ ИЗОБРАЖЕНИЙ

Цель:

- *познакомиться с основными принципами работы генеративно-состязательных сетей (GAN) и их применением в задачах генерации изображений;*
- *освоить методы предобработки наборов данных изображений, подготавливая их для обучения GAN;*
- *применить методы глубокого обучения для создания, тренировки модели GAN с использованием библиотек TensorFlow и Keras;*
- *изучить процесс генерации новых изображений на основе обученной модели, экспериментируя с различными параметрами генератора;*
- *провести анализ качества сгенерированных изображений, сравнивая их с реальными изображениями из обучающего набора;*
- *проанализировать возможные проблемы, такие как "коллапс моды" и рассмотреть методы их решения;*
- *исследовать продвинутые архитектуры GAN, такие как DCGAN, и понять их преимущества по сравнению с базовыми моделями GAN.*

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Генеративно-состязательные сети (GAN) являются одним из наиболее прорывных достижений в области глубокого обучения за последние годы. Эти нейронные сети используются для генерации данных, которые почти неотличимы от реальных, будь то изображения, музыка или текст.

Ключевым принципом работы GAN является взаимодействие двух нейронных сетей: генератора и дискриминатора. Генератор создает изображения, стараясь максимально приблизить их к настоящим, а дискриминатор пытается отличить сгенерированные изображения от реальных. По мере совместного обучения эти две сети улучшают свою работу, создавая все более качественные изображения.

Чтобы успешно работать с GAN, необходима должная предобработка данных. Изображения, которые будут использоваться для обучения, должны быть нормализованы, чтобы иметь одинаковый размер, а также могут быть подвергнуты аугментации, чтобы увеличить размер датасета и обеспечить модели разнообразные примеры для обучения.

Применение методов глубокого обучения с использованием библиотек TensorFlow и Keras сильно упрощает процесс создания и обучения модели GAN. Благодаря различным инструментам и оптимизаторам, которые предоставляют эти библиотеки, можно сосредоточиться на самой архитектуре сети, не беспокоясь о низкоуровневых деталях реализации.

После обучения модели GAN важно провести анализ качества сгенерированных изображений. Один из наиболее распространенных способов оценки

FreeBusta is knowledge without borders!

качества – визуальный анализ. Однако в научных исследованиях также используются более формальные метрики, такие как Inception Score.

Несмотря на все преимущества GAN, при их обучении могут возникать проблемы. Одной из таких проблем является "коллапс моды", когда генератор начинает создавать ограниченное количество видов изображений, игнорируя разнообразие в обучающем датасете. Чтобы бороться с этим явлением, исследователи предложили ряд методик, включая введение штрафов за однообразие генерации или использование более сложных архитектур сети.

Заключительным этапом изучения GAN является знакомство с продвинутыми архитектурами, такими как DCGAN (Deep Convolutional GAN). Эта архитектура представляет собой глубокую сверточную нейронную сеть, которая способна создавать изображения высокого разрешения с еще большим детализацией и качеством.

DCGAN преобразует понятие стандартных генеративно-состязательных сетей, внося в их архитектуру элементы сверточных нейронных сетей (CNN). Таким образом, DCGAN сочетает в себе преимущества обоих подходов, позволяя генерировать детализированные и качественные изображения. Основные принципы DCGAN включают использование сверточных слоев в генераторе и дискриминаторе, отсутствие полносвязных слоев, а также использование определенных функций активации, таких как ReLU и LeakyReLU.

Другой важной особенностью DCGAN является нормализация пакетов, что помогает улучшить стабильность обучения и сократить время, необходимое для достижения сходимости. Такая нормализация обеспечивает равномерное распределение активаций на протяжении всей нейронной сети, предотвращая искажения и ускоряя процесс обучения.

С развитием DCGAN и других архитектур, таких как Conditional GAN, StackGAN и BigGAN, возможности генеративно-состязательных сетей продолжают расширяться. Conditional GAN, например, позволяют генерировать изображения на основе определенных условий или меток, что делает их идеальными для задач, где требуется создание изображений определенной категории.

Также стоит упомянуть про применение GAN в области передачи стилей, где они используются для комбинирования стилей разных изображений или применения стилей художественных произведений к фотографиям. Это демонстрирует многообразие и гибкость GAN в области обработки изображений.

Применение GAN не ограничивается только графикой. Их используют в медицинских исследованиях для генерации медицинских изображений, в финансовой сфере для создания синтетических данных и даже в музыкальной индустрии для генерации музыкальных композиций.

С развитием технологий и методов обучения, потенциал GAN продолжает расти, обещая новые и еще более удивительные применения в будущем.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

ПРАКТИЧЕСКАЯ ЧАСТЬ

Для более наглядного представления работы GAN рассмотрим пример нейронной сети, которая генерирует изображения на основе данных MNIST, представляющих собой набор рукописных цифр. Полностью код представлен в приложении, результат выполнения работы нейронной сети, обученной на двух эпохах с размером обучающего пакета 128 единиц, представлен на рисунке 1.



Рисунок 1 – Результат обучения нейронной сети

Реализацию кода по представленному выше заданию можно разделить на несколько шагов:

1. Импорт необходимых блоков и библиотек:

```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import mnist
from keras.models import Model, Sequential
from keras.layers import Dense, LeakyReLU, BatchNormaliza-
tion, Reshape, Flatten, Input
from keras.optimizers import Adam
from keras import initializers
```

- `import numpy as np`: этот импорт подключает библиотеку NumPy и дает ей псевдоним np. NumPy – это основная библиотека для работы с массивами и математическими функциями в Python. Она используется для эффективных математических операций над массивами данных;
- `import matplotlib.pyplot as plt`: здесь импортируется модуль `pyplot` из библиотеки `matplotlib` с псевдонимом `plt`. Этот модуль предоставляет функциональность для визуализации данных, например, для построения графиков;
- `from keras.datasets import mnist`: из библиотеки `keras.datasets` импортируется набор данных `mnist`. MNIST — это база данных рукописных цифр, которая часто используется для обучения различных систем машинного обучения;

FreeBusta is knowledge without borders!

- `from keras.models import Model, Sequential`: здесь импортируются два типа моделей из библиотеки `Keras`:
  - `Model`: базовый класс для создания любой модели `Keras`;
  - `Sequential`: модель, которая подразумевает линейный стек слоев;
- `from keras.layers import Dense, LeakyReLU, BatchNormalization, Reshape, Flatten, Input`: эта строка импортирует различные слои, которые можно использовать при создании моделей в `Keras`:
  - `Dense`: полносвязный слой, каждый нейрон которого связан со всеми входами;
  - `LeakyReLU`: функция активации, которая пропускает небольшое значение при отрицательных входах, предотвращая "затухание" градиентов;
  - `BatchNormalization`: слой нормализации, который обеспечивает центрирование и масштабирование входных данных по батчам.
  - `Reshape`: слой, изменяющий форму входных данных;
  - `Flatten`: слой, преобразующий многомерный тензор в одномерный;
  - `Input`: используется для задания формы входных данных;
- `from keras.optimizers import Adam`: здесь импортируется оптимизатор `Adam` из `Keras`. `Adam` — это популярный алгоритм оптимизации, который сочетает в себе преимущества двух других алгоритмов: `AdaGrad` и `RMSProp`;
- `from keras import initializers`: из библиотеки Keras импортируются инициализаторы. Они используются для задания начального распределения весов в модели. В данном коде используется инициализатор RandomNormal, который инициализирует веса в соответствии с нормальным распределением.

2. **Подготовка данных:**
- Загрузка и предобработка данных из набора MNIST:

```
(x_train, _), (_, _) = mnist.load_data()
```

- загружаем только изображения для обучения `(x_train)`, метки классификации нас не интересуют. Каждое изображение представляет собой матрицу 28x28 пикселей:

```
x_train = (x_train.astype(np.float32) - 127.5) / 127.5
```

- следующий шаг преобразует каждый пиксель изображения из диапазона [0, 255] в диапазон [-1, 1]. Такая нормализация часто используется для улучшения сходимости обучения нейронных сетей:

```
x_train = (x_train.astype(np.float32) - 127.5) / 127.5
```

- на этом шаге изображение преобразуется в одномерный массив из 784 элементов:

```
x_train = x_train.reshape(x_train.shape[0], 784)
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

3. Определение параметров нейронной сети:

```
random_dim = 100
adam = Adam(lr=0.0002, beta_1=0.5)
```

Здесь, random_dim задает размерность входного шума для генератора, adam – это оптимизатор с предустановленными параметрами.

4. Создание архитектуры генератора: генератор принимает на вход вектор случайного шума и преобразует его в изображение.

```
def create_generator():
    generator = Sequential()
    generator.add(Dense(256, input_dim=random_dim, ker-
nel_initializer=initializers.RandomNormal(stddev=0.02)))
    generator.add(LeakyReLU(0.2))
    generator.add(BatchNormalization(momentum=0.8))
    generator.add(Dense(512))
    generator.add(LeakyReLU(0.2))
    generator.add(BatchNormalization(momentum=0.8))
    generator.add(Dense(1024))
    generator.add(LeakyReLU(0.2))
    generator.add(BatchNormalization(momentum=0.8))
    generator.add(Dense(784, activation='tanh'))
    generator.compile(loss='binary_crossentropy', opti-
mizer=adam)
    return generator
```

Здесь инициализируется слой с 256 нейронами. Важным моментом является инициализация весов с помощью нормального распределения со стандартным отклонением 0,02 – это может помочь в начальной фазе обучения.

LeakyReLU – это вариация функции активации ReLU, которая позволяет небольшим отрицательным значениям пройти через нейрон, предотвращая "затухание" градиентов.

Последний слой генератора использует функцию активации tanh, что гарантирует, что выходные значения будут в диапазоне [-1, 1].

5. Создание архитектуры дискриминатора:

```
def create_discriminator():
    discriminator = Sequential()
    discriminator.add(Dense(1024, input_dim=784, ker-
nel_initializer=initializers.RandomNormal(stddev=0.02)))
    discriminator.add(LeakyReLU(0.2))
    discriminator.add(Dense(512))
    discriminator.add(LeakyReLU(0.2))
    discriminator.add(Dense(256))
    discriminator.add(LeakyReLU(0.2))
    discriminator.add(Dense(1, activation='sigmoid'))
    discriminator.compile(loss='binary_crossentropy', op-
timizer=adam)
```


FreeBusta is knowledge
without borders!



```
    return discriminator
```

Дискриминатор рассматривает одномерные массивы (изображения) длиной 784 пикселя и классифицирует их как реальные или сгенерированные. На последнем слое используется активация "sigmoid", что позволяет получать вероятность принадлежности изображения к реальному классу.

6. Создание GAN:

```
def create_gan(discriminator, generator):
    discriminator.trainable = False
    gan_input = Input(shape=(random_dim,))
    x = generator(gan_input)
    gan_output = discriminator(x)
    gan = Model(gan_input, gan_output)
    gan.compile(loss='binary_crossentropy', opti-
mizer=adam)
    return gan
```

Данная функция соединяет генератор и дискриминатор в единую модель. При этом обучение дискриминатора отключается, чтобы обеспечить обучение только генератора при тренировке GAN.

7. Обучение GAN

```
def train_gan(epochs=1, batch_size=128):
    batch_count = x_train.shape[0] // batch_size
    generator = create_generator()
    discriminator = create_discriminator()
    gan = create_gan(discriminator, generator)
    for e in range(epochs):
        for _ in range(batch_count):
            noise = np.random.normal(0, 1,
size=[batch_size, random_dim])
            generated_images = generator.predict(noise)
            image_batch = x_train[np.random.randint(0,
x_train.shape[0], size=batch_size)]
            X = np.concatenate([image_batch, gener-
ated_images])
            y_dis = np.zeros(2 * batch_size)
            y_dis[:batch_size] = 0.9
            d_loss = discriminator.train_on_batch(X,
y_dis)
            noise = np.random.normal(0, 1,
size=[batch_size, random_dim])
            y_gen = np.ones(batch_size)
            g_loss = gan.train_on_batch(noise, y_gen)
        print(f"Epoch {e} | Discriminator Loss: {d_loss}
| Generator Loss: {g_loss}")
    plot_generated_images(generator)
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

```
    return generator, discriminator
```

Функция `train_gan` принимает два аргумента: `epochs` и `batch_size`, которые определяют количество эпох обучения и размер обучающего набора соответственно.

`batch_count = x_train.shape[0] // batch_size`: Эта строка определяет, сколько наборов для обучения в одной эпохе.

Затем создаются три модели: генератор, дискриминатор и GAN, объединяющая их вместе.

Внешний цикл `for e in range(epochs)`: итерируется по эпохам обучения.

Внутренний цикл `for _ in range(batch_count)`: итерируется по батчам в текущей эпохе. Для каждого набора:

- генерируется шумовой вектор `noise`, который передается в генератор для создания искусственных изображений;
- из обучающего датасета `x_train` случайным образом выбирается, реальный набор обучения, состоящий из изображений;
- сгенерированные и реальные изображения объединяются в один обучающий набор X;
- создается метка `y_dis` для обучения дискриминатора: первая половина имеет значение 0.9 (для реальных изображений), вторая половина – 0 (для искусственных изображений). Значение 0,9 используется вместо 1 для техники "`label smoothing`", что помогает стабилизировать обучение;
- дискриминатор обучается на наборе из реальных и сгенерированных изображений. Затем создается новый шумовой вектор, и генератор пытается обмануть дискриминатор через модель `GAN`.

После завершения каждой эпохи выводятся потери дискриминатора и генератора, а после завершения всех эпох отображаются примеры изображений, сгенерированных текущим генератором с помощью функции `plot_generated_images`.

В конце функции возвращаются обученные модели генератора и дискриминатора.

8. Визуализация и сохранение обученной модели (генератора и дискриминатора):

```
def plot_generated_images(generator, examples=10, dim=(1,
10), figsize=(10, 1)):
    noise = np.random.normal(0, 1, size=[examples, ran-
dom_dim])
    generated_images = generator.predict(noise)
    generated_images = generated_images.reshape(examples,
28, 28)
    plt.figure(figsize=figsize)
    for i in range(generated_images.shape[0]):
        plt.subplot(dim[0], dim[1], i + 1)
```


FreeBusta is knowledge
without borders!

```
        plt.imshow(generated_images[i], interpola-
tion='nearest', cmap='gray')
        plt.axis('off')
    plt.tight_layout()
    plt.show()
#Обучение:
generator, discriminator = train_gan(epochs=2,
batch_size=128)
# Сохранение обученных моделей:
generator.save('gan_generator_model.h5')
discriminator.save('gan_discriminator_model.h5')
```

Функция `plot_generated_images`:
Эта функция предназначена для визуализации изображений, которые генерирует генератор сети GAN.

`generator`: модель генератора, которая принимает случайный шум и возвращает сгенерированные изображения.

`examples`: количество изображений, которые нужно сгенерировать и отобразить.

`dim`: кортеж, определяющий размерность сетки для отображения изображений. По умолчанию отображает изображения в одну строку и 10 столбцов.

`figsize`: кортеж, определяющий физический размер фигуры (ширина x высота).

В функции выполняются следующие шаги:

- генерация случайного шума `(noise)` с помощью нормального распределения;
- с помощью модели генератора `(generator.predict(noise))` получаем сгенерированные изображения;
- изменение формы сгенерированных изображений, чтобы их можно было отобразить как 28x28 пикселей;
- создание фигуры с заданным размером `figsize`;
- в цикле отображение каждого сгенерированного изображения на сетке.
- отображение всей сетки сгенерированных изображений.

Обучение GAN:

- здесь вызывается функция `train_gan` с параметрами `epochs=2` и `batch_size=128`. Это означает, что вы обучаете GAN в течение 2 эпох, используя батчи по 128 примеров;
- функция `train_gan` возвращает обученные модели генератора и дискриминатора, которые затем сохраняются в переменные `generator` и `discriminator` соответственно.

После обучения вы сохраняете модели генератора и дискриминатора на диск в формате `.h5` для дальнейшего использования.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Источники:

1. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). Generative adversarial nets. Advances in Neural Information Processing Systems. [электронный ресурс] Доступно по адресу: https://arxiv.org/abs/1406.2661 (дата обращения: 27.08.2023).

2. Radford, A., Metz, L., & Chintala, S. (2015). Unsupervised representation learning with deep convolutional generative adversarial networks. arXiv preprint. [электронный ресурс] Доступно по адресу: https://arxiv.org/abs/1511.06434 (дата обращения: 27.08.2023).

3. Сайт Keras. Руководство по созданию генеративно-состязательных сетей с использованием Keras. [электронный ресурс] Доступно по адресу: https://keras.io/examples/generative/ (дата обращения: 27.08.2023).

4. Kingma, D. P., & Welling, M. (2013). Auto-encoding variational bayes. arXiv preprint. [электронный ресурс] Доступно по адресу: https://arxiv.org/abs/1312.6114 (дата обращения: 27.08.2023).

5. Foster, D. (2019). Generative Deep Learning: Teaching Machines to Paint, Write, Compose, and Play. O'Reilly Media. [электронный ресурс] Доступно по адресу: https://www.oreilly.com/library/view/generative-deep-learning/9781492041931/ (дата обращения: 27.08.2023).

6. Сайт Matplotlib. Руководство по использованию библиотеки Matplotlib для визуализации данных на Python. [электронный ресурс] Доступно по адресу: https://matplotlib.org/stable/contents.html (дата обращения: 27.08.2023).

Задание:

- Изучите основы генеративно-состязательных сетей (GANs) и их применение для генерации изображений. Обратите внимание на архитектуру, состоящую из генератора и дискриминатора.
- Запустите предоставленный код для генерации изображений с использованием GAN и TensorFlow. Убедитесь в его работоспособности и проанализируйте полученные результаты.
- Экспериментируйте с архитектурой дискриминатора, добавив дополнительные сверточные или полносвязные слои. Сравните полученные результаты с базовой моделью.
- Измените размерность шума, подаваемого на вход генератору. Попробуйте значения 50, 100, 150 и 200 и оцените влияние этого параметра на качество генерируемых изображений.
- Экспериментируйте с параметрами обучения: размером пакета, количеством эпох и скоростью обучения. Оцените, как изменение этих параметров влияет на результаты.
- Попробуйте различные оптимизаторы для обучения генератора и дискриминатора, например, 'RMSprop' или 'Adagrad', вместо 'Adam'. Сравните качество генерируемых изображений с разными оптимизаторами.

FreeBusta is knowledge without borders!

- Примените GAN для генерации изображений на основе другого набора данных, например, CIFAR-10. Анализируйте различия в качестве генерации между разными исходными данными.
- Оцените влияние регуляризации и методов предотвращения переобучения (например, dropout) на качество генерируемых изображений.

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Список рекомендуемых источников

1. Браунли, Джейсон. Введение в машинное обучение с помощью Python. Руководство для специалистов по работе с данными [электронный ресурс]/ Джейсон Браунли; пер. с англ. – М.: ООО «И. Д. «Вильямс», 2019. – 212 с. – URL: https://www.labirint.ru/books/595088/ (дата обращения: 23.07.2023).
2. Герон, Орельен. Прикладное машинное обучение с использованием Scikit-Learn и TensorFlow [электронный ресурс] / Орельен Герон; пер. с англ. – М.: ДМК Пресс, 2019. – 542 с.
3. Гудфеллоу, Ян. Глубокое обучение [электронный ресурс] / Ян Гудфеллоу, Йошуа Бенжио, Аарон Курвилль; пер. с англ. – М.: ДМК Пресс, 2018. – 664 с.
4. Карпатий, Андрей. Глубокое обучение. С нуля до профессионала [электронный ресурс] / Андрей Карпатий; пер. с англ. – М.: ДМК Пресс, 2019. – 490 с.
5. Макклур, Тариен. Нейронные сети. Учебник для начинающих [электронный ресурс] / Тариен Макклур. – М.: ДМК Пресс, 2020. – 310 с.
6. Николенко, Сергей. Глубокое обучение. Погружение в мир нейронных сетей [электронный ресурс] / Сергей Николенко, Алексей Кадурин, Евгений Архангельская. – М.: ДМК Пресс, 2018. – 456 с.
7. Рашка, Себастьян. Python и машинное обучение [электронный ресурс] / Себастьян Рашка; пер. с англ. – Изд. дом "Питер", 2017. – 418 с. – URL: https://coderbooks.ru/books/python/python_i_mashinnoe_obuchenie_rashka_mirdzhalili_2020/ (дата обращения 19.08.2023).
8. Сайт с открытыми курсами по глубокому обучению Deep Learning Specialization от Эндрю Ына. [электронный ресурс] режим доступа: https://www.coursera.org/specializations/deep-learning (дата обращения 19.08.2023).
9. Штернцев, Григорий. Нейронные сети: полный курс / Григорий Штернцев. – М.: БХВ-Петербург, 2018. – 528 с.
10. TensorFlow. Официальная документация и руководства. [электронный ресурс] Режим доступа: https://www.tensorflow.org/overview (дата обращения 19.08.2023).
11. Keras. Официальная документация и руководства. [электронный ресурс] URL: https://keras.io/guides/ (дата обращения 1.09.2023).
12. Neural Network Architectures. Обзор различных архитектур нейронных сетей. [электронный ресурс] Режим доступа: https://neuralnetworksanddeeplearning.com/ (дата обращения 1.09.2023).
13. Fast.ai. Курсы по глубокому обучению. [электронный ресурс] URL: https://course.fast.ai/ (дата обращения 17.04.2023).
14. Chollet, F. Глубокое обучение с помощью Python [электронный ресурс] / Франсуа Шолле; пер. с англ. – М.: ДМК Пресс, 2019. – 352 с. – URL: https://www.manning.com/books/deep-learning-with-python (дата обращения: 20.08.2023).

FreeBusta is knowledge
without borders!

15. Goodfellow, I., Глубокое обучение [электронный ресурс]/ Иан Гудфеллоу, Йошуа Бенджио, Аарон Курвилль; пер. с англ. – М.: ДМК Пресс, 2018. – 664 с. – URL: http://www.deeplearningbook.org (дата обращения: 20.08.2023).

16. James, G., Введение в статистическое обучение [электронный ресурс] / Гарет Джеймс, Даниэль Уиттен, Тревор Хасти, Роберт Тибширани; пер. с англ. – М.: ДМК Пресс, 2017. – 432 с. – URL: https://www.statlearning.com (дата обращения: 27.08.2023).

17. LeCun, Y., База данных рукописных цифр MNIST [электронный ресурс] / Ян ЛеКун, Коринна Кортес, Кристоф Бурдж; пер. с англ. – М.: ДМК Пресс, 2010. – 56 с. – URL: http://yann.lecun.com/exdb/mnist (дата обращения: 20.08.2023).

18. Сайт OpenCV. Руководство по использованию библиотеки OpenCV для обработки изображений и машинного зрения. [электронный ресурс] Режим доступа: https://opencv.org/ (дата обращения 20.08.2023).

19. Szegedy, C., Возвращаясь к первому слою: визуализация входных данных и их атрибутов [электронный ресурс] / Криштиан Сегеди и др. – Сан-Франциско: Google Inc., 2015. – 21 с. – URL: https://arxiv.org/pdf/1412.6806.pdf (дата обращения 20.08.2023).

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

Приложение
Код, применяемый в лабораторной работе №1

```
# Импортируем необходимые библиотеки
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical

# Установим параметры
batch_size = 128
num_classes = 10
epochs = 20

# Загрузим данные MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Преобразуем изображения в одномерный вектор
x_train = x_train.reshape(60000, 784)
x_test = x_test.reshape(10000, 784)

# Нормализуем данные, чтобы их было проще обрабатывать
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train /= 255
x_test /= 255

# Преобразуем метки в категориальные
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)

# Создадим модель
model = Sequential()
model.add(Dense(128, activation='relu', input_shape=(784,)))
model.add(Dense(num_classes, activation='softmax'))

# Скомпилируем модель
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])

# Обучим модель
model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs,
validation_data=(x_test, y_test))

# Проведем тестирование модели
```


FreeBusta is knowledge
without borders!



```
score = model.evaluate(x_test, y_test, verbose=0)
print('Test loss:', score[0])
print('Test accuracy:', score[1])
```

Код, применяемый в лабораторной работе №1, с реализацией определения конкретной цифры:

```
# Импортируем необходимые библиотеки
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical
import cv2
import numpy as np

# Установим параметры
batch_size = ***
num_classes = ***
epochs = ***

# Загрузим данные MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Преобразуем изображения в одномерный вектор
x_train = x_train.reshape(60000, 784)
x_test = x_test.reshape(10000, 784)

# Нормализуем данные, чтобы их было проще обрабатывать
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')
x_train /= 255
x_test /= 255

# Преобразуем метки в категориальные
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)

# Создадим модель
model = Sequential()
model.add(Dense(128, activation='relu', input_shape=(784,)))
model.add(Dense(num_classes, activation='softmax'))

# Скомпилируем модель
model.compile(loss='categorical_crossentropy', optimizer='adam',
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

```
metrics=['accuracy'])

# Обучим модель
model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs,
validation_data=(x_test, y_test))

# Проведем тестирование модели
score = model.evaluate(x_test, y_test, verbose=0)
print('Test loss:', score[0])
print('Test accuracy:', score[1])

# Здесь начинается новый блок кода

# Допустим, image_path - это путь к вашему изображению.
image_path = '4.png'


# Загружаем изображение в черно-белом формате (флаг 0)
img = cv2.imread(image_path, 0)

# Инвертируем цвета
img = cv2.bitwise_not(img)

# Изменяем размер изображения на 28x28 пикселей
img = cv2.resize(img, (28, 28))

# Преобразуем изображение в одномерный вектор и нормализуем его
img = img.reshape(1, 784).astype('float32') / 255

# Применяем модель к изображению
prediction = model.predict(img)

# Получаем индекс наиболее вероятного класса
digit = np.argmax(prediction)

print('Predicted digit:', digit)
```

Код, применяемый в лабораторной работе №2

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense
```


FreeBusta is knowledge
without borders!



```
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras.preprocessing import image

# Загрузка и предобработка данных
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0

# Загрузка сохраненной модели
#model = load_model('cifar10_model.h5')
#Создение модели
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32,
3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Если вы все еще хотите обучать модель, оставьте код обучения. В
противном случае его можно закомментировать:
model.fit(x_train, y_train, epochs=20, validation_data=(x_test,
y_test))

# Сохранение всей модели в файл HDF5
model.save('my_model.h5')


# Функция для загрузки и предобработки изображения
def load_and_preprocess_img(img_path):
    img = image.load_img(img_path, target_size=(32, 32))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    return img_array / 255.0

# Загрузка изображения
img_path = 'sh.png'
img_array = load_and_preprocess_img(img_path)
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

```
# Предсказание для изображения
predictions = model.predict(img_array)
predicted_class = np.argmax(predictions, axis=1)

class_names = ['airplane', 'car', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

# Вывод результатов
plt.imshow(image.load_img(img_path))
plt.title(f"Predicted: {class_names[predicted_class[0]]}")
plt.show()
```

Код, применяемый в лабораторной работе №3

Теоретическая часть

```
from keras.models import Sequential
from keras.layers import LSTM, Dense
model = Sequential()
model.add(LSTM(128, input_shape=(timesteps, data_dim)))
model.add(Dense(10, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

Практическая часть

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Загружаем текстовый файл и приводим его к нижнему регистру
text = open("text.txt", "r", encoding="utf-8").read().lower()

# Создаем список уникальных символов и два словаря для их индекса-
ции
chars = sorted(list(set(text)))
char_indices = dict((c, i) for i, c in enumerate(chars))
indices_char = dict((i, c) for i, c in enumerate(chars))

# Устанавливаем длину последовательности и шаг для создания обуча-
ющих данных
maxlen = 40
step = 3
sequences = []
next_chars = []
```


FreeBusta is knowledge
without borders!



```
# Создаем последовательности и следующие символы для обучения
for i in range(0, len(text) - maxlen, step):
    sequences.append(text[i: i + maxlen])
    next_chars.append(text[i + maxlen])

# Инициализация массивов для входных данных (X) и меток (y)
X = np.zeros((len(sequences), maxlen, len(chars)), dtype=np.bool_)
y = np.zeros((len(sequences), len(chars)), dtype=np.bool_)

# Заполняем X и y данными: X будет представлять собой закодирован-
ные последовательности, y - следующие символы
for i, sequence in enumerate(sequences):
    for t, char in enumerate(sequence):
        X[i, t, char_indices[char]] = 1
    y[i, char_indices[next_chars[i]]] = 1

# Создаем модель с LSTM слоем и полносвязным слоем на выходе
model = Sequential()
model.add(LSTM(128, input_shape=(maxlen, len(chars))))
model.add(Dense(len(chars), activation='softmax'))

# Компилируем модель с категориальной кросс-энтропией и оптимиза-
тором adam
model.compile(loss='categorical_crossentropy', optimizer='adam')

# Обучаем модель
model.fit(X, y, batch_size=128, epochs=10)

# Функция для генерации текста на основе начального текста
def generate_text(seed_text, length=400):
    generated = seed_text
    for i in range(length):
        x_pred = np.zeros((1, maxlen, len(chars)))
        for t, char in enumerate(seed_text):
            x_pred[0, t, char_indices[char]] = 1.

        preds = model.predict(x_pred, verbose=0)[0]
        next_index = np.argmax(preds)
        next_char = indices_char[next_index]

        seed_text = seed_text[1:] + next_char
        generated += next_char
    return generated
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

```
# Выбираем случайный стартовый индекс для генерации и выводим сге-
нерированный текст
start_index = np.random.randint(0, len(text) - maxlen - 1)
seed_text = text[start_index: start_index + maxlen]
print(generate_text(seed_text))
```

Код, применяемый в лабораторной работе №4

```
import numpy as np
import matplotlib.pyplot as plt
from keras.datasets import mnist
from keras.models import Model, Sequential
from keras.layers import Dense, LeakyReLU, BatchNormalization, Re-
shape, Flatten, Input
from keras.optimizers import Adam
from keras import initializers

#Загрузка и предобработка данных:
(x_train, _), (_, _) = mnist.load_data()
x_train = (x_train.astype(np.float32) - 127.5) / 127.5
x_train = x_train.reshape(x_train.shape[0], 784)

#Параметры:
random_dim = 100
adam = Adam(lr=0.0002, beta_1=0.5)

#Создание генератора:
def create_generator():
    generator = Sequential()
    generator.add(Dense(256, input_dim=random_dim, kernel_initial-
izer=initializers.RandomNormal(stddev=0.02)))
    generator.add(LeakyReLU(0.2))
    generator.add(BatchNormalization(momentum=0.8))
    generator.add(Dense(512))
    generator.add(LeakyReLU(0.2))
    generator.add(BatchNormalization(momentum=0.8))
    generator.add(Dense(1024))
    generator.add(LeakyReLU(0.2))
    generator.add(BatchNormalization(momentum=0.8))
    generator.add(Dense(784, activation='tanh'))
    generator.compile(loss='binary_crossentropy', optimizer=adam)
    return generator

#Создание дискриминатора:
def create_discriminator():
    discriminator = Sequential()
```


FreeBusta is knowledge
without borders!



```
    discriminator.add(Dense(1024, input_dim=784, kernel_initial-
izer=initializers.RandomNormal(stddev=0.02)))
    discriminator.add(LeakyReLU(0.2))
    discriminator.add(Dense(512))
    discriminator.add(LeakyReLU(0.2))
    discriminator.add(Dense(256))
    discriminator.add(LeakyReLU(0.2))
    discriminator.add(Dense(1, activation='sigmoid'))
    discriminator.compile(loss='binary_crossentropy', opti-
mizer=adam)
    return discriminator

#Создание GAN
def create_gan(discriminator, generator):
    discriminator.trainable = False
    gan_input = Input(shape=(random_dim,))
    x = generator(gan_input)
    gan_output = discriminator(x)
    gan = Model(gan_input, gan_output)
    gan.compile(loss='binary_crossentropy', optimizer=adam)
    return gan

#Обучение GAN:
def train_gan(epochs=1, batch_size=128):
    batch_count = x_train.shape[0] // batch_size

    generator = create_generator()
    discriminator = create_discriminator()
    gan = create_gan(discriminator, generator)

    for e in range(epochs):
        for _ in range(batch_count):
            noise = np.random.normal(0, 1, size=[batch_size, ran-
dom_dim])
            generated_images = generator.predict(noise)
            image_batch = x_train[np.random.randint(0,
x_train.shape[0], size=batch_size)]

            X = np.concatenate([image_batch, generated_images])
            y_dis = np.zeros(2 * batch_size)
            y_dis[:batch_size] = 0.9

            d_loss = discriminator.train_on_batch(X, y_dis)
            noise = np.random.normal(0, 1, size=[batch_size, ran-
dom_dim])
            y_gen = np.ones(batch_size)
```


Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta

```
            g_loss = gan.train_on_batch(noise, y_gen)

        print(f"Epoch {e} | Discriminator Loss: {d_loss} | Genera-
tor Loss: {g_loss}")

    plot_generated_images(generator)

    return generator, discriminator
#Функция для отображения сгенерированных изображений:
def plot_generated_images(generator, examples=10, dim=(1, 10),
figsize=(10, 1)):
    noise = np.random.normal(0, 1, size=[examples, random_dim])
    generated_images = generator.predict(noise)
    generated_images = generated_images.reshape(examples, 28, 28)
    plt.figure(figsize=figsize)
    for i in range(generated_images.shape[0]):
        plt.subplot(dim[0], dim[1], i + 1)
        plt.imshow(generated_images[i], interpolation='nearest',
cmap='gray')
        plt.axis('off')
    plt.tight_layout()
    plt.show()
#Обучение:
generator, discriminator = train_gan(epochs=2, batch_size=128)
# Сохранение обученных моделей:
generator.save('gan_generator_model.h5')
discriminator.save('gan_discriminator_model.h5')
```


FreeBusta is knowledge
without borders!



Учебное издание

Терлецкий Алексей Сергеевич
Терлецкая Екатерина Сергеевна

НЕЙРОННЫЕ СЕТИ И ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ: ОСНОВЫ НЕЙРОННЫХ СЕТЕЙ НА ЯЗЫКЕ PYTHON

Учебно-методическое пособие

Редактор В.И. Буланова
Подписано в печать 25.12. 2023 г.
Формат 60 x 84 1/16.
Бумага для копировальной техники.
4,5 п. л. Тираж 300 экземпляров.
Заказ № 2046. .

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Липецкий государственный педагогический университет
имени П.П. Семенова-Тян-Шанского»
398020, г. Липецк, ул. Ленина, 42

Отпечатано в отделе редакционно-печатной деятельности
ЛГПУ имени П.П. Семенова-Тян-Шанского
398020, г. Липецк, ул. Ленина, 42

Забери welcome-бонус
2000 USDT
по промокоду
FreeBusta